



DEPARTEMENT INFORMATIQUE
Institut Universitaire de Technologie

8, rue Montaigne
56000 Vannes cedex

WEIS Marin

Année universitaire 2025 - 2026

Rapport de STAGE de fin d'année

« Contribution au développement de *Sam*,
l'assistant IA conversationnel de recommandation
de produits reconditionnés de Combak »

Stage effectué au sein de l'entreprise
COMBAK



Du 13 avril au 17 juin 2026

Maitre de stage : M. Tom ROUCHY, co-fondateur & CEO de Combak

Entreprise : Combak, Identity 1, 9D rue de Châtillon, 8^e étage, 35000 Rennes

Enseignant tuteur : M. Gauthier QUILAN, IUT de Vannes, UBS

Table des matières

Remerciements.....	2
1. Introduction.....	3
2. Présentation de Combak et de l'environnement de stage.....	5
2.1. Activité de l'entreprise	5
2.2. Modèle économique et enjeux techniques.....	6
2.3. Équipe, encadrement et rôle confié.....	7
2.4. Environnement technique de travail.....	9
3. Sujet du stage et analyse de l'existant.....	10
3.1. Situation à mon arrivée.....	10
3.2. Données et outils préexistants	11
3.3. Problématique du stage.....	12
3.4. Cahier des charges et objectifs	13
4. Démarche de travail et choix de conception	16
4.1. Organisation du stage et méthode de travail.....	17
4.2. Contraintes principales.....	19
4.3. Choix d'une architecture ancrée dans les données.....	19
4.4. Choix liés à l'intelligence artificielle générative	20
4.5. Limites et arbitrages.....	20
5. Modélisation avant développement.....	21
5.1. Architecture générale de Sam.....	21
5.2. Flux d'un message utilisateur.....	22
5.3. Modèle d'état conversationnel.....	23
5.4. Modèle de données utilisé.....	24
5.5. Garde-fous prévus avant codage.....	25
6. Travail réalisé.....	25
6.1. Refonte de l'interface conversationnelle.....	26
6.2. Route API et streaming des réponses.....	27
6.3. Function calling et accès aux données catalogue	27
6.4. Mémoire conversationnelle.....	28
6.5. Garde-fous anti-hallucination	29
6.6. Backoffice de suivi et analytics.....	30
6.7. Tests, documentation et état du logiciel	31
7. Résultats obtenus.....	33

7.1. Livrables réalisés	33
7.2. Comparaison avec les objectifs initiaux	34
7.3. Évolutions proposées	36
7.4. Limites restantes	36
8. Acquis de compétences	37
8.1. Compétences techniques	37
8.2. Compétences méthodologiques	38
8.3. Compétences humaines	38
8.4. Formation acquise pendant le stage	39
8.5. Difficultés rencontrées	39
9. Conclusion	40
10. Usage de l'intelligence artificielle chez Combak	41
Glossaire	42
Table des figures	43
Annexes	44
Annexe 1 — Tableau chronologique simplifié des principales contributions	44
Annexe 2 — Exemples de scénarios de test Sam	45
Annexe 3 — Confidentialité	45
Annexe 4 — Tableau détaillé des commits	46
Résumé / Abstract et mots-clés	46
Résumé	47
Abstract	47

Remerciements

Je tiens à remercier en premier lieu Tom Rouchy, cofondateur et CEO de Combak, pour la confiance qu'il m'a accordée en m'intégrant au développement de Sam, l'assistant IA de l'entreprise, et en me confiant la responsabilité de plusieurs chantiers à part entière. Sa disponibilité, la qualité de ses revues de code et sa pédagogie sur l'ingénierie des modèles de langage et le pilotage produit ont fait de ce stage un véritable terrain d'apprentissage.

Je remercie également l'ensemble de l'équipe de Combak pour son accueil au sein de cette jeune startup, en particulier Kim Ung, Faustine Honoré et Hazel Rozanski, pour la bonne ambiance de travail, l'entraide et les échanges au quotidien.

Je remercie enfin l'équipe pédagogique de l'IUT de Vannes pour la préparation et le suivi de ce stage, et tout particulièrement Gauthier Quilan, mon enseignant tuteur, pour son accompagnement et sa disponibilité.

1. Introduction

Dans le cadre de ma deuxième année de BUT Informatique à l'IUT de Vannes, j'ai effectué mon stage de fin d'année au sein de Combak, une startup rennaise qui développe un comparateur de produits reconditionnés* et de seconde main. La plateforme couvre aujourd'hui l'électronique et l'électroménager, et a pour ambition de s'étendre à d'autres familles de produits (vélos, vêtements...). Ce stage s'est déroulé du 13 avril au 17 juin 2026.

Combak évolue dans un contexte où l'achat de produits reconditionnés répond à plusieurs enjeux. Pour les utilisateurs, il permet d'accéder à des appareils souvent moins chers que le neuf. Pour l'environnement, il contribue à prolonger la durée de vie des produits électroniques, dont la fabrication représente une part importante de leur impact écologique. Cependant, le marché du reconditionné reste parfois difficile à comprendre : les offres sont nombreuses, les prix changent régulièrement, les états des produits varient selon les marchands, et les utilisateurs ne savent pas toujours quel modèle choisir selon leur usage.

L'objectif de Combak est de simplifier cette recherche. La plateforme agrège des offres issues de plusieurs marchands partenaires, les rattache à des modèles produits, puis les rend comparables sur une même interface. L'utilisateur peut ainsi consulter différents critères comme le prix, l'état du produit, la garantie ou le vendeur.

Ma mission principale a consisté à contribuer au développement de Sam, aussi appelé Ask Combak, l'assistant IA conversationnel de Combak. Cet assistant a pour objectif d'accompagner l'utilisateur dans sa recherche de produits reconditionnés. Au lieu de parcourir uniquement des listes d'offres ou d'utiliser des filtres classiques, l'utilisateur peut formuler un besoin en langage naturel, par exemple : « Je cherche un smartphone à moins de 400 € avec une bonne autonomie ». Sam doit alors comprendre cette demande, interroger les données réelles du catalogue Combak et proposer une réponse claire, fiable et utile.

Le projet Sam existait déjà avant mon arrivée. Mon rôle n'a donc pas été de créer entièrement l'assistant, mais de contribuer à son évolution, à sa fiabilisation et à son industrialisation. J'ai travaillé sur plusieurs aspects : l'interface conversationnelle, l'expérience mobile, la mémoire conversationnelle, le fonction calling*, les garde-fous anti-hallucination*, le backoffice* d'analyse, le tracking* analytics et les tests.

En parallèle, j'ai également participé à un second chantier important : l'industrialisation du déploiement de nouvelles catégories produit. L'objectif était de transformer l'ajout d'une catégorie en un processus plus reproductible, en travaillant sur les parsers, le matching par regex, les facettes*, le SEO*, le maillage interne et l'intégration dans le catalogue. Plusieurs catégories ont été livrées pendant la période analysée, notamment les casques audio, les machines à café, les lave-linge, les sèche-linge et les airfryers.

Ce stage m'a donc permis de travailler sur un projet concret, intégré à une plateforme existante et soumis à des contraintes réelles : fiabilité des données, expérience utilisateur, performance, coût des modèles d'intelligence artificielle, qualité du code et suivi produit. Il m'a également amené à comprendre qu'un développement professionnel ne se limite pas à

écrire du code : il faut aussi analyser l'existant, faire des choix, tester, mesurer, documenter et prévoir la maintenance.

Ce rapport présente d'abord l'entreprise Combak et son environnement technique. Il détaille ensuite le sujet du stage, les données préexistantes, la problématique, le cahier des charges initial et les objectifs attendus. Les parties suivantes présenteront ma démarche de travail, les choix de conception, la modélisation réalisée avant le développement, le travail effectué, les résultats obtenus et les compétences acquises.

Remarque : les termes techniques suivis d'un astérisque () sont définis dans le glossaire, en fin de rapport.*

2. Présentation de Combak et de l'environnement de stage

2.1. Activité de l'entreprise

Combak est une startup rennaise spécialisée dans la comparaison de produits reconditionnés et d'occasion. L'ambition est de couvrir à terme l'ensemble des catégories de produits : la plateforme se concentre aujourd'hui sur l'électronique et l'électroménager, avec une extension prévue vers d'autres familles de produits (vélos, vêtements...). Le site permet aux utilisateurs de rechercher un produit, comme un smartphone, un ordinateur portable, une tablette, une montre connectée, un casque audio ou un appareil électroménager, puis de comparer les offres disponibles chez différents marchands.

Le principe de Combak repose sur l'agrégation d'offres partenaires. Pour un même modèle, la plateforme récupère des offres issues de marchands comme Fnac, Darty, Cdiscount, Back Market, AfB ou d'autres vendeurs spécialisés. Ces offres proviennent de flux d'affiliation*, puis sont traitées afin d'être rattachées à des modèles produits comparables.

L'intérêt pour l'utilisateur est de centraliser l'information. Au lieu de comparer manuellement les offres sur plusieurs sites marchands, il peut consulter une fiche modèle regroupant les prix, les états, les garanties et les vendeurs disponibles. Cette approche permet de rendre le marché du reconditionné plus lisible.

Le rôle de Combak n'est pas de vendre directement les produits. La plateforme redirige l'utilisateur vers les marchands partenaires grâce à des liens trackés. Combak agit donc comme un intermédiaire entre l'utilisateur et les vendeurs, avec un objectif de simplification, de comparaison et d'aide à la décision.

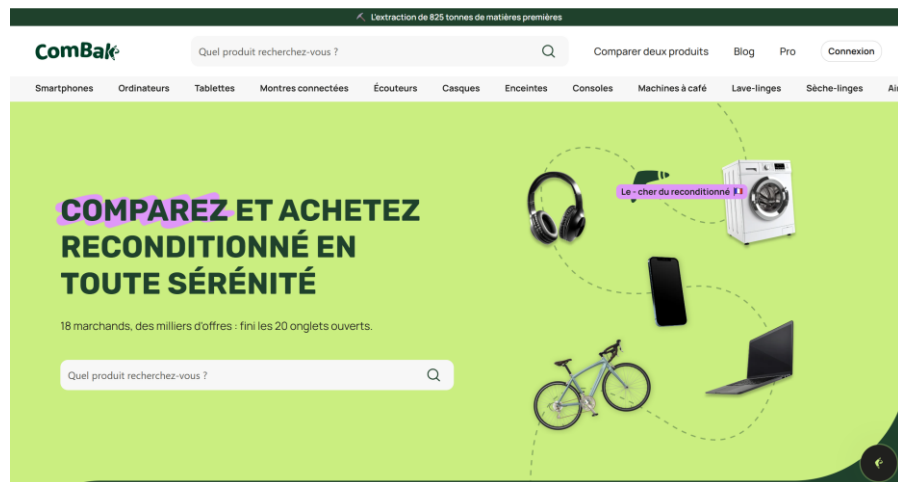


Figure 1 — Page d'accueil de Combak, donnant accès aux différents univers de produits reconditionnés.

Combak couvre plusieurs catégories de produits. Pendant mon stage, j'ai notamment participé à l'ajout ou à l'amélioration de catégories aussi variées que l'audio ou l'électroménager. Ce point est important, car il montre que la plateforme n'est pas limitée aux smartphones. Elle doit être capable de gérer des produits très différents, avec des critères propres à chaque catégorie.

2.2. Modèle économique et enjeux techniques

Le modèle économique de Combak repose principalement sur l'affiliation. Pour cela, la plateforme s'appuie sur des régies d'affiliation externes comme Awin, Kwanko ou Effiliation, qui fournissent les liens trackés vers les marchands et remontent les conversions. Lorsqu'un utilisateur clique sur une offre, il est redirigé vers le site du marchand via ce lien tracké ; s'il réalise un achat, la conversion est attribuée à Combak par la régie, qui reverse une commission. Ce fonctionnement rend le suivi particulièrement important, car il faut pouvoir relier une visite, un clic et une conversion qui se produit en partie en dehors du site.

Le suivi des événements combine donc plusieurs briques. Combak dispose d'un tracking first-party*, c'est-à-dire d'un système de mesure reposant sur ses propres données (un cookie de suivi et des événements stockés directement en base), utilisé notamment pour l'analyse produit et le pilotage de Sam. À cela s'ajoutent des services externes : des outils de web analytics et de suivi publicitaire comme Google Analytics ou Meta, ainsi que les régies d'affiliation pour la remontée des conversions. L'enjeu technique est de réconcilier ces différentes sources pour relier de façon fiable une visite, un clic et un achat.

Le trafic de Combak repose aussi en grande partie sur le référencement naturel. Les utilisateurs peuvent arriver sur le site via des articles ou des pages optimisées pour le SEO, puis poursuivre vers le comparateur. L'un des enjeux est donc de transformer un utilisateur en phase d'information en utilisateur prêt à comparer ou à cliquer sur une offre.

Sam s'inscrit dans cette logique : l'assistant doit guider l'utilisateur plus rapidement vers une recommandation adaptée.

D'un point de vue technique, Combak doit résoudre plusieurs problèmes. Le plus fondamental est la diversité des données partenaires : chaque marchand fournit ses offres dans un format différent, avec des noms, des prix, des états et des garanties qui ne suivent pas la même structure, ce qui oblige à parser, nettoyer et normaliser ces flux. Vient ensuite le rattachement des offres aux bons modèles, car une même référence peut s'écrire de plusieurs façons selon les marchands (une marque avec ou sans apostrophe, une casse différente, des variantes dans le nom). Pendant mon stage, certains travaux ont justement porté sur la normalisation des marques et le matching regex*, pour rendre le système plus tolérant à la ponctuation.

S'ajoutent deux exigences. La performance de recherche, d'abord : l'utilisateur doit obtenir vite des résultats pertinents, ce que permet Typesense* avec ses requêtes rapides et ses filtres à facettes (prix, marque, état, catégorie). La fiabilité des recommandations, ensuite, qui devient critique avec Sam et fait l'objet de la partie 3.3.

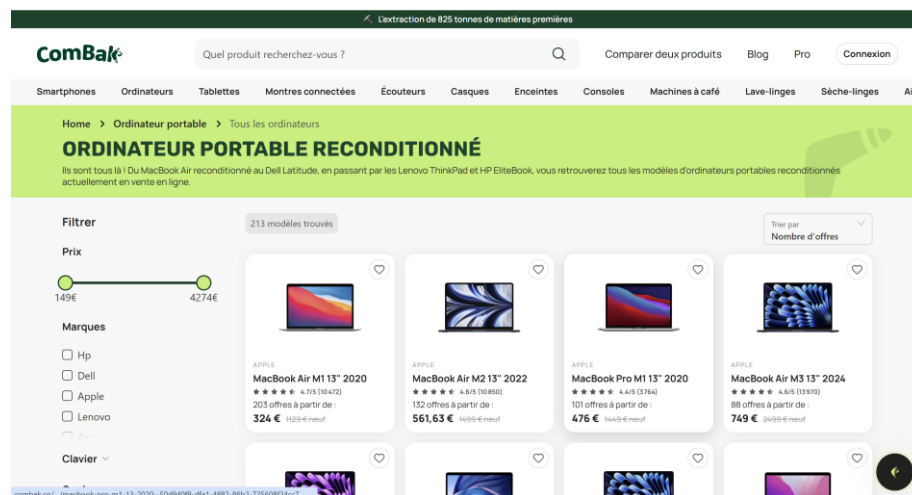


Figure 2 — Page d'une catégorie Combak (ordinateurs portables reconditionnés) avec filtres et offres comparées.

2.3. Équipe, encadrement et rôle confié

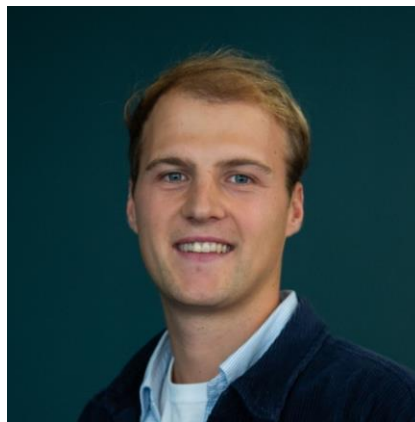
J'ai effectué mon stage en présentiel, dans les bureaux de Combak à Rennes. L'entreprise est installée au sein de Novapuls, un accélérateur de startups du quartier d'affaires d'Eurorennes, juste à côté de la gare. Travailler sur place, dans cet environnement entrepreneurial et au sein d'une petite équipe, m'a permis d'échanger très facilement avec mon maître de stage et de suivre l'avancement du produit au quotidien, notamment lors des points réguliers.



Les bureaux de Combak, au sein de l'accélérateur Novapuls (Eurorennes, Rennes).

Combak est une petite équipe, ce qui m'a permis d'être rapidement impliqué dans des sujets concrets. L'organisation d'une startup impose souvent une grande polyvalence : les sujets techniques, produit, SEO, analytics et business sont liés. Un choix technique peut avoir un impact direct sur l'expérience utilisateur, sur le référencement ou sur la conversion.

Mon maître de stage était Tom Rouchy, cofondateur et CEO de Combak, et aussi le développeur principal de la plateforme. Ingénieur de formation et ancien business developer, il a cofondé Combak en 2024 avec l'idée de rendre le marché du reconditionné plus lisible. C'est lui qui avait initié le projet Sam avant mon arrivée, et il a assuré mon encadrement au quotidien : compréhension de l'architecture existante, choix techniques, priorités produit et revues de code. Être encadré directement par le fondateur, qui connaissait le projet dans ses moindres détails, a rendu les retours rapides et concrets.



Tom Rouchy, cofondateur et CEO de Combak, mon maître de stage.

L'équipe comprenait aussi des personnes travaillant sur d'autres aspects de l'entreprise, notamment le SEO et la communication. Même si mon rôle était principalement technique, cet environnement m'a permis de comprendre que le développement d'une fonctionnalité ne peut pas être séparé du contexte produit. Par exemple, l'ajout d'une catégorie ne

consiste pas seulement à afficher de nouveaux produits : il faut aussi penser aux pages SEO, au maillage interne, aux facettes, à la navigation et au suivi des performances.

Mon rôle était celui d'un stagiaire développeur full-stack, avec une forte dimension liée à l'intelligence artificielle appliquée. J'ai travaillé sur le front-end, le back-end, les données, l'analytics et les outils internes. Plusieurs chantiers m'ont été confiés avec autonomie, tout en étant régulièrement relus et orientés par mon maître de stage.

2.4. Environnement technique de travail

La plateforme Combak repose sur une stack web moderne, principalement orientée TypeScript. Le framework utilisé est Next.js* 15, avec une combinaison du Pages Router et de l'App Router pour certaines parties, notamment le streaming* du chat. Le langage principal est TypeScript, avec une configuration stricte permettant de limiter les erreurs de typage. L'interface utilise React, Ant Design et des modules SCSS.

Côté données, Combak utilise MongoDB* et Typesense. MongoDB sert notamment à stocker des offres, des règles de matching, des données analytics et des conversations liées au chatbot. Typesense est utilisé comme moteur de recherche pour les collections de modèles et d'offres. Il permet d'effectuer des recherches rapides avec des filtres adaptés aux pages catalogue et aux outils de Sam.

L'hébergement est assuré par Vercel*. Cette solution permet de déployer l'application et certaines tâches planifiées, notamment via des fonctions serverless et des crons. Ce choix est cohérent avec une petite équipe, car il facilite le déploiement et réduit la gestion d'infrastructure.

Le projet Sam s'appuie sur plusieurs modèles de langage, chacun pour la tâche à laquelle il est le plus adapté. La conversation et le choix des outils reposent principalement sur Gemini 2.5 Flash, rapide et économique. Les traitements plus structurés, comme l'extraction de l'état de la conversation ou la génération de titres, ainsi qu'un rôle de secours, sont confiés à Claude Haiku 4.5. L'assistant s'appuie par ailleurs sur une architecture à outils (tool calling*), détaillée en partie 4.3.

Le pipeline de données aboutit à un catalogue indexé dans Typesense, qui alimente les pages du site, les filtres, les cards produits* et les recommandations de Sam (Figure 3).

Côté outils de développement, j'ai travaillé avec l'éditeur Cursor, assisté par Claude Code (détaillé au paragraphe 10). Le code était versionné avec Git et GitHub (branches, pull requests et revues de code), les dépendances gérées avec pnpm sur Node 22, et la qualité contrôlée par des commandes comme pnpm type-check (vérification TypeScript) et pnpm lint.

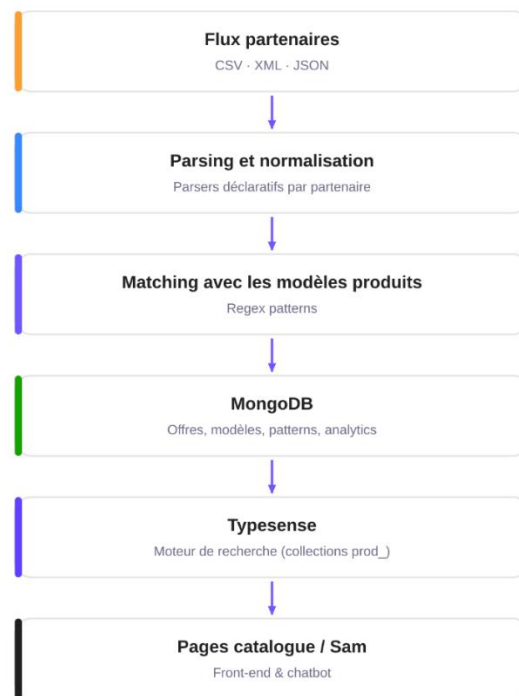


Figure 3 — Pipeline simplifié des données Combak, depuis les flux partenaires jusqu'au catalogue et à Sam.

3. Sujet du stage et analyse de l'existant

3.1. Situation à mon arrivée

À mon arrivée chez Combak, le projet Sam existait déjà sous une première forme. L'assistant avait été initié avant mon stage afin d'expérimenter une nouvelle manière d'accompagner les utilisateurs dans leur recherche de produits reconditionnés. Il disposait déjà de certaines bases, notamment une première interface conversationnelle, un fonctionnement en streaming et une intégration avec certaines fonctionnalités de la plateforme.

Mon rôle n'a donc pas été de concevoir Sam entièrement depuis le départ. La première étape de mon stage a été de comprendre l'existant : la structure du code, le fonctionnement du catalogue, les données disponibles, la logique du chatbot et les objectifs produit définis avant mon arrivée.

Cette prise en main était nécessaire car le projet était intégré à une base de code réelle. Contrairement à un projet scolaire, il ne suffisait pas de développer une fonctionnalité isolée. Chaque modification devait respecter les conventions du projet, s'intégrer dans l'architecture existante et éviter d'introduire des régressions.

J'ai rapidement compris que Sam n'était pas un simple chatbot. L'assistant devait comprendre une demande utilisateur, garder le contexte de la conversation, interroger le catalogue, afficher des produits pertinents et éviter les réponses inventées. Il devait aussi être observable par l'équipe, c'est-à-dire mesurable et analysable après son utilisation par de vrais utilisateurs.

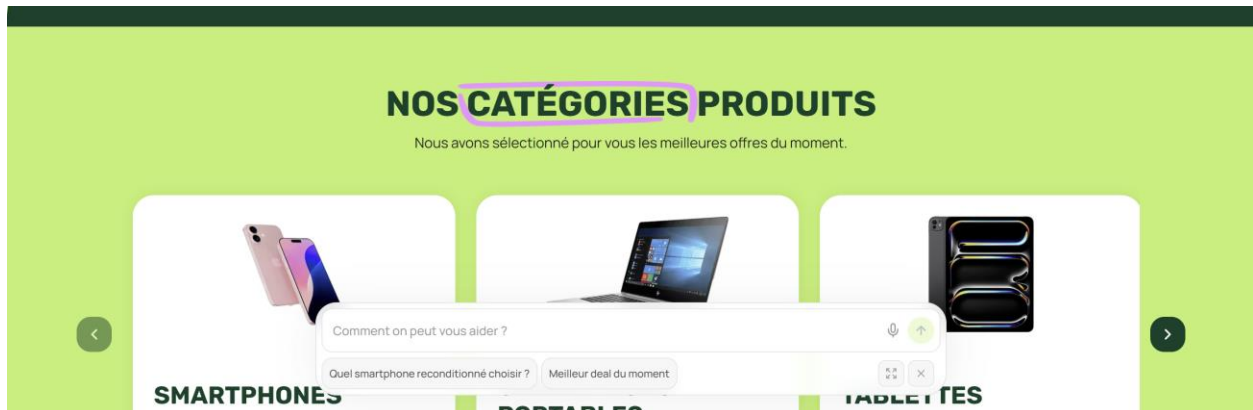


Figure 4 — Déclencheur Ask Combak intégré au site Combak.

Cette situation de départ a orienté mon travail. Il ne s'agissait pas uniquement d'ajouter de nouvelles fonctionnalités, mais aussi de rendre l'assistant plus robuste, plus fiable, plus agréable à utiliser et plus facilement améliorable.

3.2. Données et outils préexistants

Plusieurs éléments étaient déjà disponibles au début de mon stage.

Il y avait d'abord la plateforme Combak elle-même, avec ses pages catalogue, ses pages produits, ses filtres, son moteur de recherche et son pipeline de traitement des offres ; les modèles et les offres y étaient déjà indexés dans Typesense. La base MongoDB, de son côté, stockait les données nécessaires au fonctionnement du site : offres, règles de matching, analytics, conversations du chatbot et autres données internes.

Il y avait surtout une première version de Sam, déjà fonctionnelle (assistant conversationnel, streaming des réponses, appel à des outils), accompagnée d'un document de cadrage produit qui en fixait la vision et les objectifs.

Enfin, l'outillage de développement présenté en 2.4 (Git, GitHub, branches, pull requests, contrôles npm type-check, lint et test) était déjà en place, tout comme l'architecture à outils sur laquelle repose Sam.

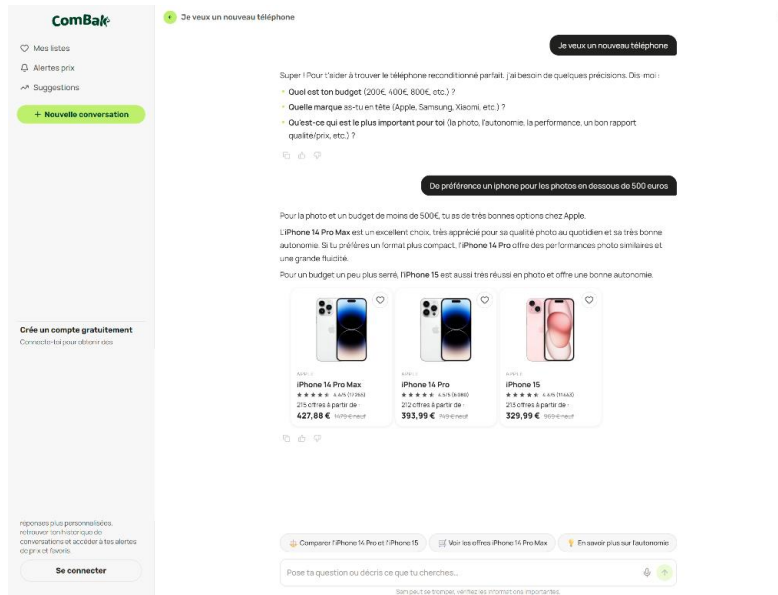


Figure 5 — Conversation avec Sam : qualification du besoin puis recommandation de smartphones reconditionnés.

3.3. Problématique du stage

Le projet Sam répond à plusieurs difficultés bien réelles rencontrées par les utilisateurs du reconditionné. La plus évidente est la surcharge d'information : pour un même modèle, il peut exister des dizaines d'offres, à des prix, des états et des garanties différents, et l'utilisateur s'y perd. À cela s'ajoute un manque de repères techniques, car beaucoup ne savent pas quelle capacité de stockage leur suffit, ni ce que vaut tel état de reconditionnement. Il y a aussi la marche entre s'informer et acheter : on arrive souvent par un article de blog, puis il faut reformuler sa recherche, filtrer et comparer, ce qui crée de la friction. Et dans le cas d'un assistant IA, une difficulté devient critique : la fiabilité. Une réponse fluide peut donner une fausse impression de justesse, alors que Sam ne doit jamais inventer un produit, un prix, un état, un marchand ou une disponibilité.

La problématique de mon stage peut donc être formulée ainsi :

Comment faire évoluer un assistant IA conversationnel de recommandation afin qu'il soit utile pour l'utilisateur, fiable dans ses réponses, connecté aux données réelles du catalogue et exploitable par ComBak dans une logique de conversion ?

Cette problématique implique plusieurs exigences. Sam doit comprendre des demandes formulées en langage naturel, conserver le contexte d'une conversation, appeler les bons outils pour rechercher dans le catalogue, produire une réponse claire, afficher des produits pertinents, limiter les hallucinations et permettre à l'équipe de suivre son usage.

Elle implique aussi une contrainte d'extensibilité. L'assistant ne devait pas être conçu uniquement pour une seule catégorie de produits. Il fallait donc construire une logique

suffisamment générique pour ne pas coder en dur une catégorie comme « smartphone » ou « laptop ».

3.4. Cahier des charges et objectifs

Le cahier des charges initial de Sam définissait l'assistant comme un outil de recommandation conversationnel intégré au site Combak. L'objectif n'était pas de créer un assistant généraliste, mais un assistant spécialisé dans l'aide au choix de produits reconditionnés.

L'utilisateur devait pouvoir accéder facilement à Sam depuis le site, par exemple grâce à un déclencheur visible ou une barre de saisie. Une fois la conversation ouverte, il devait pouvoir formuler son besoin naturellement, sans connaître à l'avance les filtres disponibles.

Sam devait ensuite qualifier la demande. Cela signifie qu'il devait comprendre les critères importants pour l'utilisateur : catégorie recherchée, budget, usage prévu, préférences de marque, contraintes techniques ou niveau d'état souhaité. Si la demande était trop vague, l'assistant devait pouvoir poser une question de précision plutôt que lancer immédiatement une recherche peu pertinente.

Le cahier des charges prévoyait également l'utilisation du catalogue Combak. Les recommandations devaient être fondées sur les données disponibles dans Typesense et MongoDB, et non sur les connaissances générales du modèle de langage. Cette contrainte était essentielle pour éviter les hallucinations.

L'assistant devait aussi afficher des réponses enrichies. Dans certains cas, une réponse textuelle ne suffit pas. Sam devait pouvoir présenter des cards produits, des comparaisons, des suggestions d'actions ou des informations complémentaires utiles à la décision d'achat.

Le projet devait enfin être observable : l'équipe devait pouvoir suivre et analyser l'usage réel de Sam une fois en ligne.

Besoin	Description
Accès simple	Permettre à l'utilisateur d'ouvrir facilement Sam depuis le site
Interface conversationnelle	Permettre une demande en langage naturel
Qualification du besoin	Identifier le budget, l'usage, la catégorie et les préférences
Accès aux données catalogue	Rechercher dans les modèles et offres réels de Combak
Fiabilité	Éviter les produits, prix, états ou marchands inventés
Réponses enrichies	Afficher du texte, des suggestions, des produits ou comparaisons

Expérience mobile	Rendre l'assistant utilisable sur smartphone
Backoffice	Permettre à l'équipe de suivre les conversations et métriques
Extensibilité	Préparer Sam à fonctionner sur plusieurs catégories

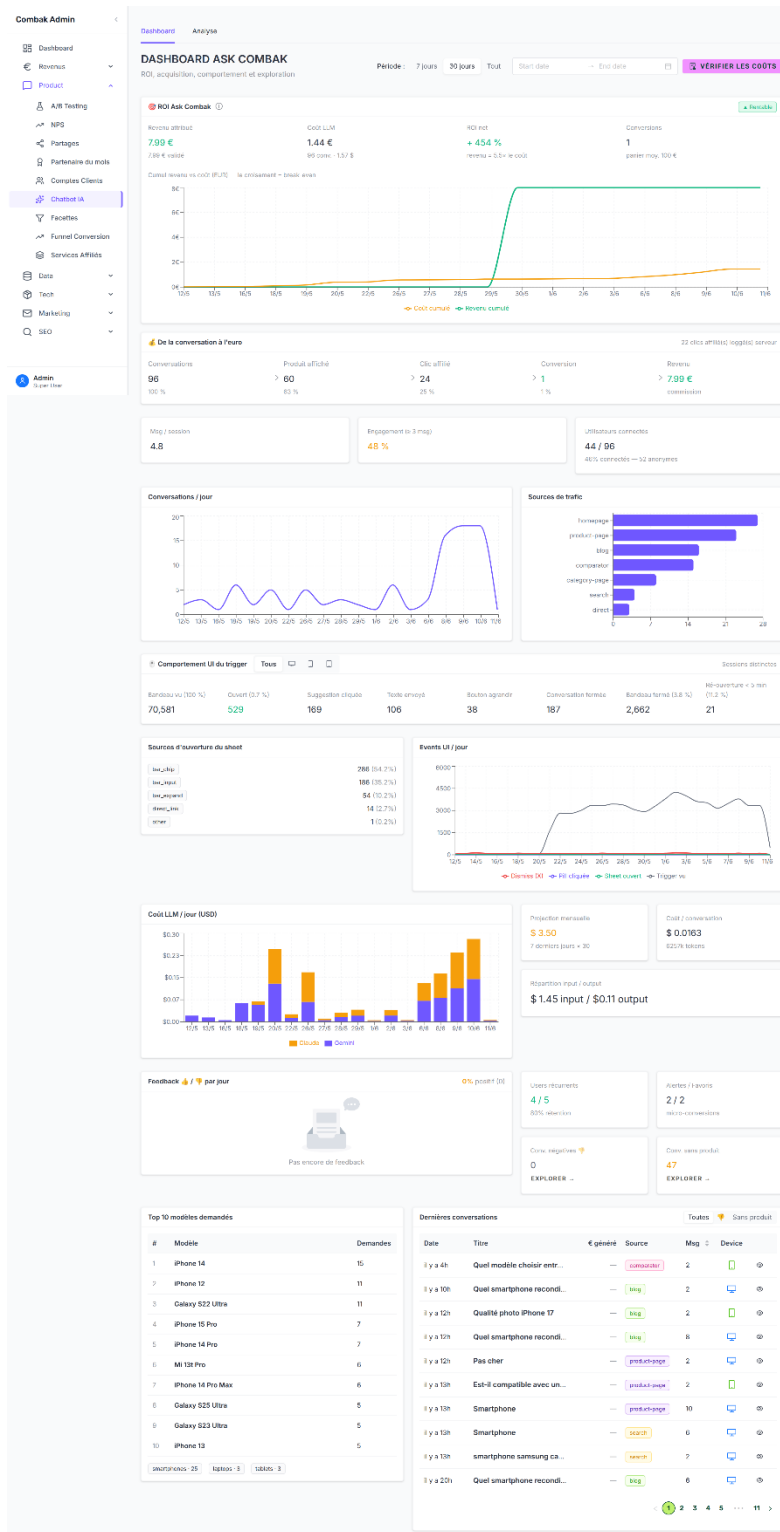


Figure 6 — Dashboard backoffice de Sam : KPIs conversationnels, funnel d'engagement et suivi des coûts.

Les objectifs attendus pendant mon stage étaient à la fois techniques, fonctionnels et méthodologiques.

Le plus visible de ces objectifs était d'améliorer l'expérience utilisateur de Sam : une interface plus claire, une meilleure adaptation au mobile, des suggestions plus visibles et des réponses plus lisibles. En parallèle, il fallait fiabiliser ces réponses en les ancrant sur le catalogue et en limitant les hallucinations, ce qui supposait plusieurs mécanismes : grounding* validator, cohérence entre les cards et les modèles cités, nomenclature stricte des grades*, ciblage exact des modèles et contrôle des budgets.

Un autre objectif était de rendre l'assistant plus générique, car Combak ne se limite pas aux smartphones : Sam devait fonctionner sur plusieurs catégories, d'où les refontes category-agnostic*. S'y ajoutait la mise en place d'un suivi produit, pour comprendre comment les utilisateurs interagissent avec lui (ouverture du chat, actions, recommandations, device, coût des modèles, clics, points de blocage). Mon stage devait aussi contribuer à l'industrialisation du déploiement de catégories (parsers, matching, regex, facettes, SEO, maillage interne) et maintenir un bon niveau de qualité de code : contributions lisibles, testées, intégrées, avec des commits* structurés et la prise en compte des revues.

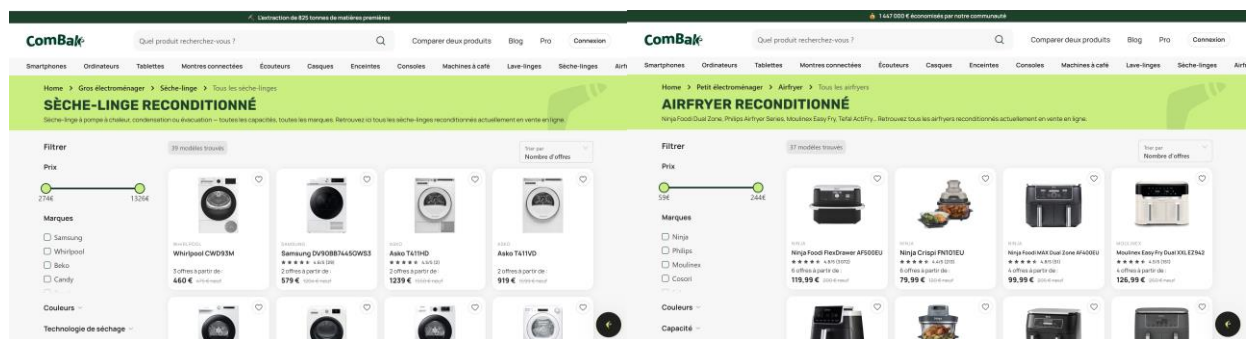


Figure 7 — Deux catégories déployées pendant le stage : sèche-linge et airfryer reconditionnés.

En résumé, mon stage avait pour but de contribuer à deux évolutions structurantes pour Combak : d'une part, rendre Ask Combak plus fiable, plus fluide et plus observable ; d'autre part, améliorer la capacité de la plateforme à intégrer de nouvelles catégories de produits. Ces deux axes sont liés par un même enjeu : aider l'utilisateur à trouver plus facilement un produit reconditionné adapté à son besoin, tout en rendant le système maintenable et mesurable pour l'entreprise.

4. Démarche de travail et choix de conception

4.1. Organisation du stage et méthode de travail

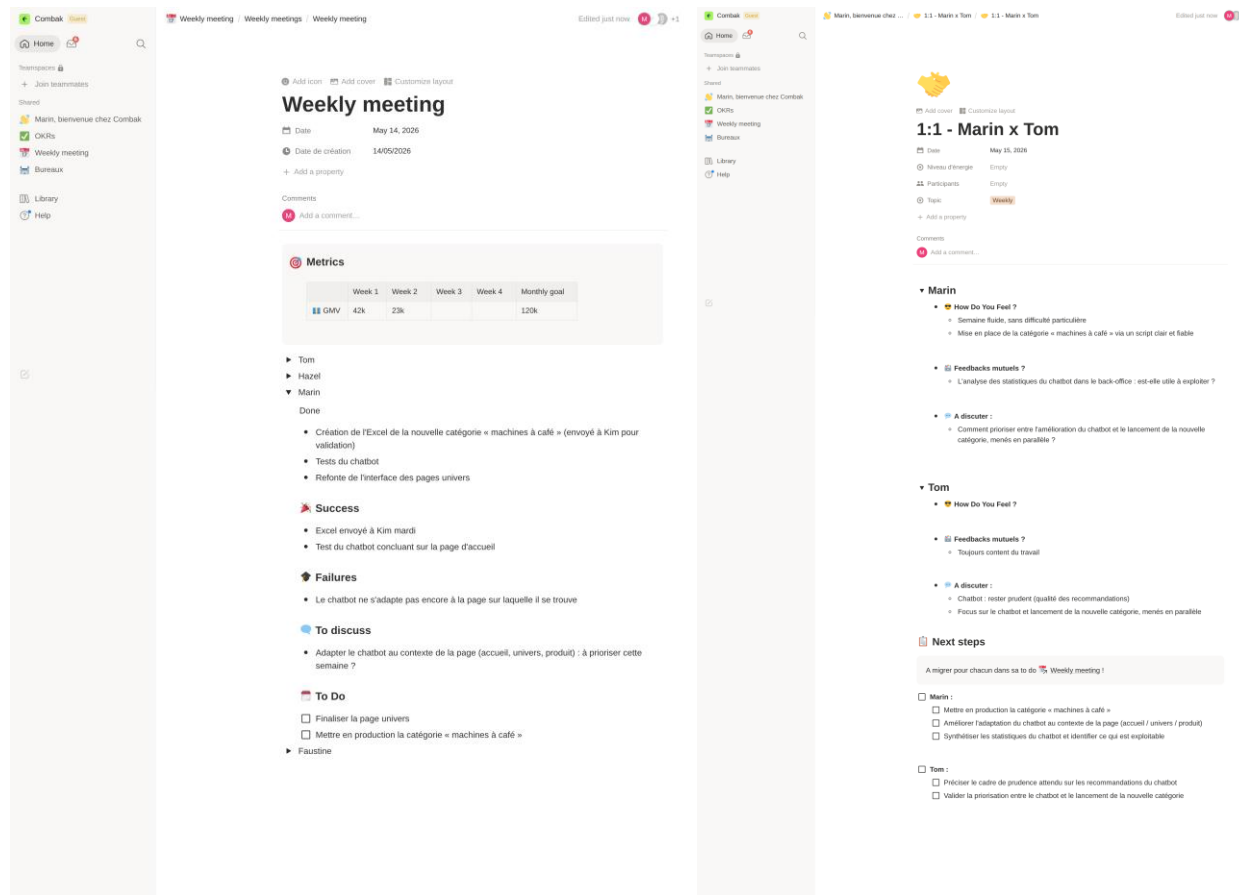
Mon stage s'est organisé autour de deux grands chantiers menés en parallèle. Le premier, et le plus important, concernait le développement de Sam, l'assistant IA conversationnel de Combak. Le second portait sur l'industrialisation du déploiement de nouvelles catégories de produits dans le comparateur. À ces deux axes se sont ajoutées des contributions transverses, comme l'intégration d'un nouveau partenaire, l'amélioration de pages catalogue, le suivi analytics et des corrections d'interface.

Au début du stage, une phase de prise en main du projet a été nécessaire (détaillée en 3.1) avant d'entrer dans le développement.

Le travail s'est ensuite fait de manière itérative. Chaque fonctionnalité était découpée en tâches plus petites, développée progressivement, testée, puis revue avec mon maître de stage. Cette méthode était particulièrement adaptée à un projet utilisant l'intelligence artificielle, car le comportement d'un assistant conversationnel ne peut pas toujours être prévu à l'avance. Une modification de prompt*, de garde-fou ou d'outil peut changer la manière dont l'assistant répond. Il fallait donc tester régulièrement, observer les résultats, corriger, puis recommencer.

Concrètement, je travaillais sur des branches dédiées (tests du chatbot, déploiements de catégories), intégrées au dépôt principal après revue. Cette organisation m'a fait comprendre le fonctionnement d'un projet logiciel collaboratif, avec ses contraintes de qualité et de maintenance.

Le suivi de mon activité reposait sur un rythme hebdomadaire structuré, tenu sous Notion. Chaque lundi, un point d'équipe permettait à chacun de partager son avancement, ses réussites, ses difficultés et ses tâches de la semaine, au regard des objectifs de l'entreprise ; il était suivi d'un entretien individuel avec mon maître de stage, centré sur mon ressenti, des retours mutuels et les prochaines étapes. Ce cadre m'a permis de rendre compte régulièrement de mon travail et d'ajuster mes priorités.



Le suivi hebdomadaire sous Notion : la réunion d'équipe (à gauche) et le point individuel avec mon maître de stage (à droite).

J'ai également participé à la mise en place d'artefacts de pilotage pour Ask Combak : brief, file de tâches, journal de décisions et résumés d'itérations. Ces documents permettaient de garder une trace des choix effectués et d'éviter que les décisions importantes restent uniquement dans des échanges informels.



Figure 8 — Organisation chronologique des principaux chantiers du stage.

4.2. Contraintes principales

Le développement de Sam devait composer avec des contraintes parfois contradictoires. La plus structurante était la fiabilité : puisqu'il s'agit d'une aide à l'achat, l'assistant ne pouvait pas inventer un prix, un produit ou une disponibilité, sous peine de nuire à la confiance et à la crédibilité du comparateur. C'est elle qui a orienté les choix techniques vers le fonction calling et les garde-fous anti-hallucination. Venait ensuite la diversité des données, car les flux marchands sont hétérogènes : noms, prix, états et garanties varient d'un flux à l'autre, et il fallait en tenir compte aussi bien dans la recherche que dans les recommandations.

L'expérience utilisateur imposait elle aussi ses règles, surtout sur mobile où l'espace est compté : il a fallu arbitrer en permanence entre richesse d'affichage et lisibilité, ce qui a justifié le travail sur le déclencheur, le drawer*, les suggestions et l'historique. S'ajoutaient le coût des modèles, qu'il fallait suivre et maîtriser, et la généricité, c'est-à-dire concevoir une logique category-agnostic plutôt que codée en dur pour une seule catégorie. Enfin, tout cela devait rester maintenable et compréhensible par l'équipe après mon départ, ce qui m'a poussé à soigner le découpage en modules, les conventions, les tests et la documentation.

4.3. Choix d'une architecture ancrée dans les données

Dictée par la contrainte de fiabilité (4.2), la décision centrale a été de connecter Sam aux données réelles de Combak plutôt que de le laisser répondre à partir de ses seules connaissances générales.

Pour cela, l'assistant repose sur une architecture à outils, aussi appelée tool calling. Le principe est de permettre au modèle de langage d'appeler des fonctions métier déterministes. Ces fonctions peuvent rechercher des modèles dans Typesense, récupérer

des offres, comparer des produits, extraire des préférences ou obtenir des informations liées aux partenaires.

Cette approche permet de séparer deux responsabilités. Le modèle de langage sert à comprendre la demande et à formuler une réponse naturelle. Les outils, eux, fournissent les données factuelles. Ainsi, lorsqu'un utilisateur demande un produit selon un budget, Sam ne doit pas inventer une réponse. Il doit interroger le catalogue, récupérer des résultats, puis construire une réponse à partir de ces données.

Ce choix améliore la fiabilité, mais il impose aussi de bien concevoir les outils. Leurs paramètres doivent être structurés, leurs résultats doivent être exploitables et les erreurs doivent être anticipées. Un outil mal conçu peut produire une mauvaise recommandation, même si le modèle de langage est performant.

4.4. Choix liés à l'intelligence artificielle générative

L'intelligence artificielle générative est utilisée dans Sam pour comprendre les demandes formulées en langage naturel et produire des réponses claires. Cependant, elle n'est pas utilisée seule. Elle est intégrée dans une architecture contrôlée, avec des outils, des règles et des validations.

Le modèle de langage intervient principalement dans trois situations : comprendre l'intention de l'utilisateur, extraire les préférences exprimées et formuler une réponse lisible. Par exemple, si l'utilisateur écrit : « Je cherche un ordinateur portable pour les cours, pas trop cher et assez léger », Sam doit comprendre plusieurs éléments : la catégorie recherchée, l'usage prévu, la contrainte de prix et le critère de mobilité.

Pour éviter de dépendre entièrement du modèle, une partie du routage est déterministe. Cela signifie que certaines décisions ne sont pas laissées uniquement à l'IA. Cette architecture hybride permet d'améliorer la robustesse du système.

Les modèles utilisés ne servent pas tous exactement au même rôle. Certains traitements sont conversationnels, tandis que d'autres sont plus structurés, comme l'extraction d'état ou la détection de contradictions. Cette séparation permet d'adapter l'usage de l'IA selon le besoin : génération de texte, analyse d'intention, extraction d'informations ou fallback.

Le streaming a également été un choix d'expérience important, pour donner une impression de rapidité dès les premiers mots ; son fonctionnement est détaillé en partie 6.2.

Enfin, l'utilisation de l'IA a nécessité une réflexion sur les limites du système. Sam ne doit pas répondre à tout. Lorsqu'une demande sort du périmètre de Combak, ou lorsque les données disponibles sont insuffisantes, l'assistant doit répondre avec prudence. Il vaut mieux qu'il demande une précision ou qu'il signale une limite plutôt qu'il invente une réponse.

4.5. Limites et arbitrages

Plusieurs arbitrages ont été nécessaires pendant le stage. Le plus important portait sur la liberté laissée au modèle : un assistant très libre paraît plus naturel mais invente plus facilement, alors qu'un assistant très contraint est plus fiable mais moins fluide. Comme Sam intervient dans une aide à l'achat, j'ai donné la priorité à la fiabilité. La qualification du besoin a demandé elle aussi des essais : fallait-il lancer une recherche dès le premier indice, ou poser d'abord quelques questions ? J'ai testé plusieurs approches avant de trouver un équilibre.

D'autres arbitrages étaient plus structurels : le choix d'une logique category-agnostic représentait davantage de travail au départ, mais il permet de réutiliser Sam sur d'autres catégories plutôt que de la recoder pour chacune.

5. Modélisation avant développement

5.1. Architecture générale de Sam

Avant de contribuer au développement de Sam, j'ai dû comprendre puis modéliser son architecture. Cette étape était importante pour identifier les responsabilités de chaque couche et éviter de modifier une partie du système sans comprendre ses dépendances.

L'architecture générale peut être découpée en trois niveaux.

Au premier niveau, l'interface utilisateur regroupe le déclencheur Ask Combak, la fenêtre de conversation, l'affichage des messages, les suggestions, les cards produits et des vues secondaires comme l'historique ou la comparaison ; elle est développée avec React, Next.js, TypeScript, Ant Design et des modules SCSS. Au-dessus, la route API* de chat reçoit les messages, prépare le contexte, applique les vérifications nécessaires, appelle le modèle de langage et renvoie la réponse en streaming : c'est le point de passage principal entre le front-end et l'orchestration IA. Enfin, la couche de services porte la logique métier de Sam (état conversationnel, outils, validation des réponses, recherche, suivi du coût, rate limiting, stockage des conversations) et s'appuie sur Typesense pour le catalogue et MongoDB pour la persistance.

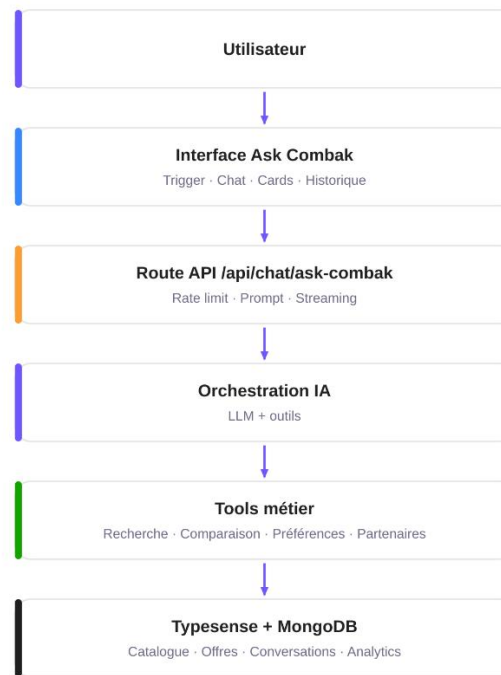


Figure 9 — Architecture générale de Sam : interface, route API, modèles IA, outils et données.

5.2. Flux d'un message utilisateur

Pour mieux comprendre le fonctionnement de Sam, j'ai modélisé le flux d'un message utilisateur. Cette modélisation permet de suivre le parcours d'une demande depuis l'interface jusqu'à la réponse finale.

Lorsqu'un utilisateur envoie un message, celui-ci est d'abord transmis à la route API. L'API vérifie la validité de la requête, applique les limites nécessaires et prépare le contexte de conversation. Ce contexte contient notamment les messages précédents, l'état conversationnel et les consignes système.

Le modèle de langage analyse ensuite la demande. Si la demande nécessite des données produit, il peut appeler un outil. Par exemple, une demande comme « Je cherche un laptop reconditionné pour les cours à moins de 700 € » nécessite une recherche dans le catalogue. L'outil correspondant interroge Typesense et retourne des résultats structurés.

Ces résultats sont ensuite utilisés pour produire une réponse en langage naturel. La réponse est affichée progressivement grâce au streaming. Si des produits sont recommandés, ils peuvent être affichés sous forme de cards adaptées à l'interface du chat.

Enfin, certaines informations sont enregistrées : message, conversation, outils appelés, coût, actions utilisateur et éventuelles métriques utiles pour le backoffice.

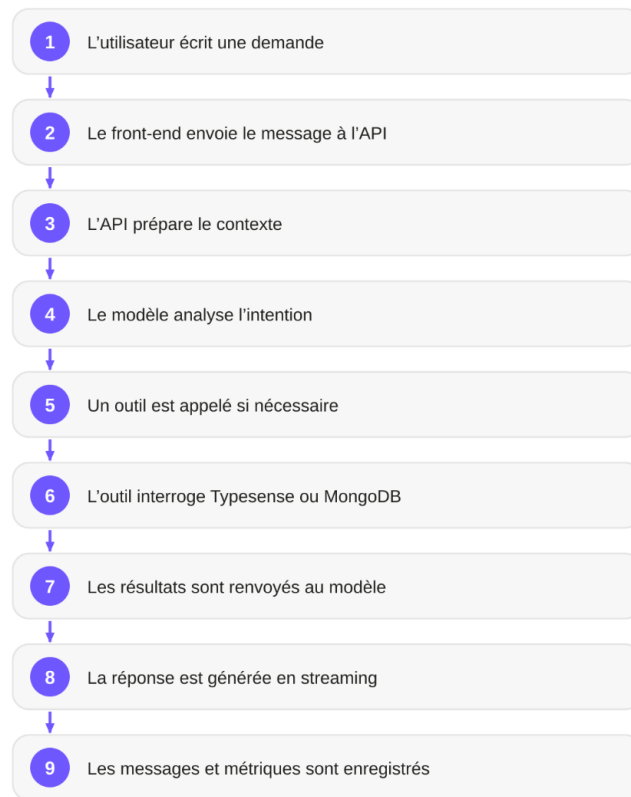


Figure 10 — Flux de traitement d'un message utilisateur dans Sam.

5.3. Modèle d'état conversationnel

Un assistant conversationnel doit être capable de conserver le contexte d'une discussion. Sans mémoire, chaque message serait traité séparément. L'utilisateur devrait alors répéter ses contraintes à chaque fois, ce qui rendrait l'expérience peu naturelle.

Le modèle d'état conversationnel sert à retenir les informations utiles à la recommandation. Il ne s'agit pas de stocker toute la conversation de manière brute, mais d'identifier les éléments importants : catégorie recherchée, budget, usage prévu, préférences, contraintes techniques ou produits déjà proposés.

Dans le cas de Sam, cette mémoire devait rester simple. Une structure trop complexe aurait été difficile à maintenir et aurait pu créer des incohérences. À l'inverse, une mémoire trop pauvre ne permet pas une conversation fluide. L'objectif était donc de trouver un équilibre entre simplicité et utilité.

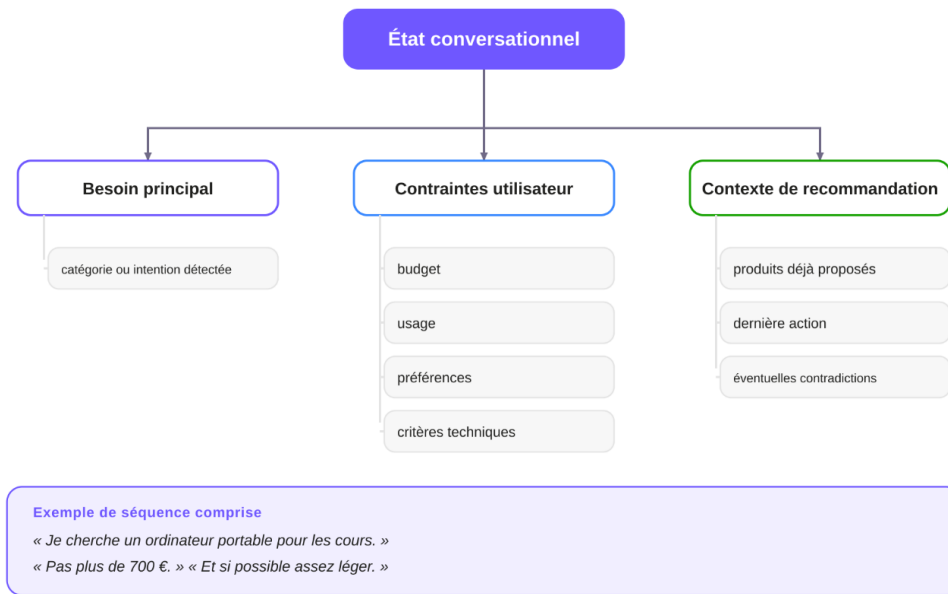


Figure 11 — Exemple simplifié d'état conversationnel utilisé par Sam.

Sam doit comprendre que les trois messages décrivent le même besoin. La mémoire conversationnelle permet donc de construire progressivement une recommandation pertinente.

5.4. Modèle de données utilisé

Sam s'appuie sur plusieurs types de données.

Les données produits décrivent d'abord les modèles du catalogue : nom, marque, catégorie, caractéristiques principales et informations utiles à la comparaison. Les données d'offres y sont rattachées : pour un même modèle, plusieurs offres peuvent exister chez différents marchands, avec le prix, l'état, le marchand, la garantie et le lien affilié. Viennent ensuite les données conversationnelles (messages, historique, état de la conversation, outils appelés, métadonnées de suivi) et les données analytics, qui servent à mesurer l'usage de Sam : ouverture du chat, interactions, clics, device, coût des modèles, actions et progression dans le funnel. Une dernière catégorie concerne le backoffice, qui permet à l'équipe de consulter les conversations, d'identifier les erreurs et de prioriser les améliorations.

Type de donnée	Exemple	Utilité
Modèle produit	MacBook Air, iPhone, machine à café	Identifier le produit recherché
Offre partenaire	Prix, état, marchand, garantie	Proposer des offres réelles

Conversation	Messages utilisateur et assistant	Conserver l'échange
État conversationnel	Budget, usage, préférences	Comprendre le contexte
Analytics	Ouverture, clic, device, coût	Mesurer l'usage
Backoffice	Replay, métriques, actions	Améliorer Sam

Ce modèle montre que la qualité de Sam dépend à la fois de l'IA et de la qualité des données disponibles. Un bon modèle de langage ne suffit pas si les données produits sont incomplètes ou mal structurées.

5.5. Garde-fous prévus avant codage

Avant de développer ou d'améliorer les garde-fous, j'ai listé les principaux risques d'un assistant IA dans un comparateur. Le plus évident était l'invention de produits ou de prix : Sam pouvait recommander un modèle absent du catalogue, ou citer un tarif qui n'existait pas, alors que le prix est justement un critère de décision majeur. Venait ensuite la mauvaise interprétation des états de reconditionnement, dont les grades varient selon les marchands et appellent une nomenclature claire.

Deux autres risques étaient plus subtils. D'abord l'incohérence entre le texte et l'interface : si Sam parle d'un modèle mais affiche une autre card, l'utilisateur est induit en erreur. Ensuite la gestion des demandes ambiguës : quand l'utilisateur n'en dit pas assez, l'assistant ne doit pas inventer ses préférences, mais poser une question ou proposer plusieurs pistes.

Risque identifié	Garde-fou prévu
Produit inventé	Vérifier que le produit provient du catalogue
Prix inventé	Utiliser uniquement les prix retournés par les outils
Mauvais grade	Utiliser une nomenclature stricte
Card incohérente	Aligner les produits mentionnés et affichés
Budget incohérent	Contrôler les bornes min/max
Demande ambiguë	Poser une question de qualification
Données insuffisantes	Répondre avec prudence
Question hors périmètre	Recentrer sur l'aide à l'achat Combak

Cette phase de réflexion avant codage était importante. Dans un projet IA, les erreurs ne viennent pas seulement du code classique. Elles peuvent aussi venir des données, des prompts, des outils, du format de sortie ou de l'interprétation du modèle.

6. Travail réalisé

6.1. Refonte de l'interface conversationnelle

L'un de mes premiers chantiers a porté sur l'interface conversationnelle de Sam. L'objectif était de rendre l'assistant plus visible, plus simple à utiliser et mieux adapté aux usages mobiles.

J'ai travaillé sur le déclencheur du chat, qui permet à l'utilisateur d'ouvrir Sam depuis le site. Ce déclencheur devait être suffisamment visible pour encourager l'usage, sans gêner la navigation. La refonte a notamment pris la forme d'une barre de saisie flottante et d'une page dédiée à Ask Combak.

J'ai aussi travaillé sur l'affichage de la conversation. Une interface de chatbot doit être lisible : les messages de l'utilisateur et de l'assistant doivent être clairement distingués, les réponses doivent être faciles à lire et les actions possibles doivent être visibles. Dans le cas de Sam, l'interface devait aussi pouvoir afficher des éléments enrichis, comme des suggestions ou des cards produits.

L'adaptation mobile a été un point important. Sur un écran réduit, l'espace disponible est limité. Il fallait donc adapter la fenêtre de chat, les boutons, les menus, le scroll et l'affichage horizontal de certains éléments.



Figure 12 — Interface mobile de Sam après refonte de l'expérience conversationnelle.

6.2. Route API et streaming des réponses

J'ai également travaillé sur la route API utilisée par Sam, dont le rôle dans le parcours d'un message a été présenté en 5.1 et 5.2.

Le streaming est une partie essentielle de cette expérience. Sans streaming, l'utilisateur devrait attendre que toute la réponse soit générée avant de voir quoi que ce soit. Avec le streaming, la réponse apparaît progressivement, ce qui rend l'assistant plus fluide et plus agréable à utiliser.

La route API ne se contente pas de transmettre un message au modèle. Elle doit aussi appliquer plusieurs traitements : limitation du nombre de requêtes, nettoyage de l'entrée utilisateur, construction du prompt système, ajout du contexte conversationnel, gestion des outils et transformation de la sortie.

Ce travail m'a permis de mieux comprendre le rôle d'une API dans une application moderne. Dans un projet comme Sam, la route API est à la fois un point d'entrée, un orchestrateur et une couche de protection.

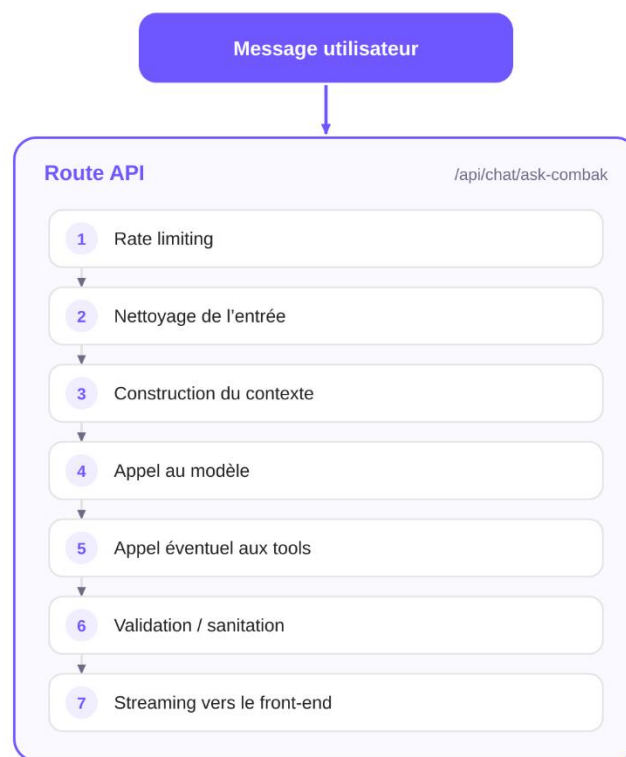


Figure 13 — Rôle de la route API dans le traitement d'un message Sam.

6.3. Function calling et accès aux données catalogue

Le function calling (présenté en 4.3) est l'un des éléments les plus importants du fonctionnement de Sam.

Dans le cadre de mon stage, j'ai contribué à l'utilisation et à l'amélioration de ces outils. Leur objectif est de connecter Sam au catalogue Combak. Lorsqu'un utilisateur exprime un besoin, l'assistant peut appeler un outil de recherche pour obtenir des modèles ou des offres correspondant aux critères.

Plutôt que de produire une liste de produits de mémoire, le modèle s'appuie sur les résultats retournés par les outils. Par exemple, si l'utilisateur cherche un produit avec un budget précis, l'outil de recherche filtre les offres ou modèles disponibles selon ce budget.

Les outils doivent être conçus avec précision. Ils doivent recevoir des paramètres clairs, produire des résultats structurés et gérer les cas où aucune donnée pertinente n'est trouvée. Cette structuration est indispensable pour que le modèle puisse utiliser correctement les résultats.

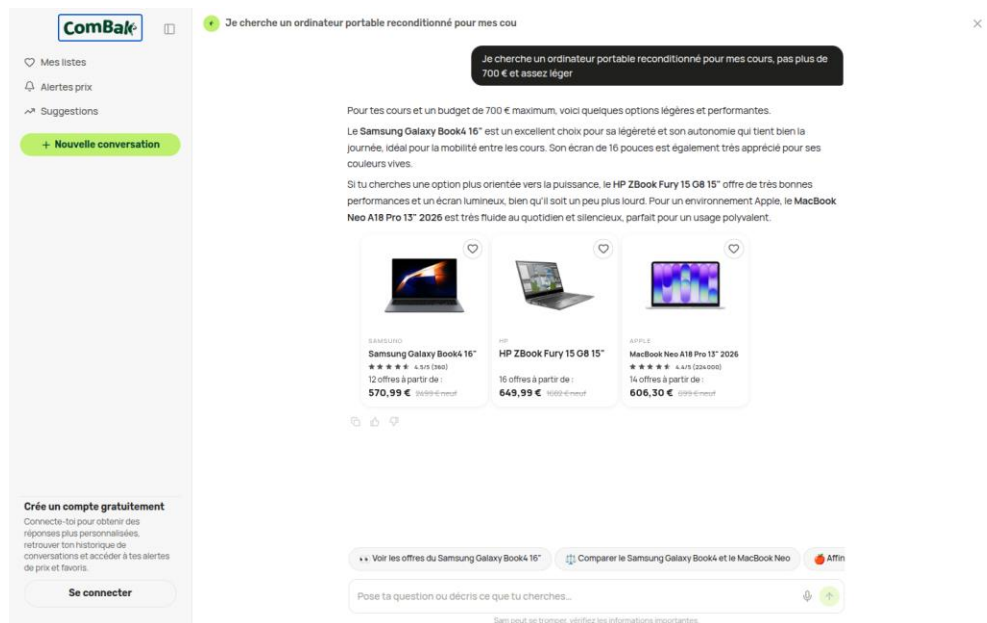


Figure 14 — Recommandation d'ordinateurs portables par Sam, avec des cards issues du catalogue réel.

6.4. Mémoire conversationnelle

J'ai participé à l'amélioration de la mémoire conversationnelle de Sam. Cette mémoire permet à l'assistant de conserver les informations utiles au fil de l'échange.

Ma contribution a porté sur la fusion de l'état conversationnel au fil des messages et sur la détection de certaines incohérences : par exemple, si l'utilisateur demande un produit très récent avec un budget très bas, l'assistant adapte sa réponse ou pose une question.

Dans la pratique, j'ai gardé cette logique d'état volontairement légère plutôt que de multiplier les champs : c'est ce qui la rend facile à maintenir et limite les incohérences au fil de l'échange.

Ce travail m'a permis de comprendre que la mémoire d'un assistant IA n'est pas seulement un historique de messages. C'est une représentation structurée de ce qui est important pour répondre correctement.

À titre d'illustration, voici une version simplifiée de la fusion de l'état conversationnel :

```
// Fusion idempotente : une nouvelle valeur n'ecrase l'ancienne
// que si elle est definie (?? = operateur de coalescence).
function mergeState(prev, next) {
  const s = {
    minBudget:      next.minBudget      ?? prev.minBudget,
    maxBudget:      next.maxBudget      ?? prev.maxBudget,
    requiredReseller: next.requiredReseller ?? prev.requiredReseller,
    usageAxes:      next.usageAxes      ?? prev.usageAxes,
  };
  // Garde-fou de coherence : on remet les bornes dans l'ordre
  if (s.minBudget != null && s.maxBudget != null && s.minBudget > s.maxBudget) {
    [s.minBudget, s.maxBudget] = [s.maxBudget, s.minBudget];
  }
  return s;
}
```

Extrait simplifié (TypeScript) : fusion de l'état à quatre champs, avec garde-fou de cohérence du budget.

6.5. Garde-fous anti-hallucination

La mise en place de garde-fous anti-hallucination a été l'un des chantiers les plus importants du stage. Dans le contexte de Sam, une hallucination peut prendre plusieurs formes : produit inexistant, prix inventé, caractéristique incorrecte, mauvaise garantie ou incohérence entre le texte et les cards affichées.

Pour limiter ces risques, plusieurs mécanismes se combinent. Le principe de base est le *grounding* : l'assistant doit s'appuyer sur les données renvoyées par les outils. À cela s'ajoutent la validation de certaines sorties et des règles inscrites dans le prompt système. Un dernier mécanisme veille à l'alignement entre les produits cités dans la réponse et ceux réellement affichés sous forme de cards, pour éviter qu'un modèle mentionné ne corresponde pas à la card montrée.

Le principe du grounding-validator peut, lui, se résumer ainsi :

```
// Anti-hallucination : on retire toute mention d'un modele qui
// n'est pas reellement present dans les resultats des outils.
function stripUngrounded(reply, allowedModels) {
  const allowed = new Set(allowedModels.map(normalize));
  return reply
    .split('\n')
    .filter(line => {
      const cited = extractModelNames(line); // modeles cites dans la ligne
      // on garde la ligne si TOUS les modeles cites sont autorises
      return cited.every(name => allowed.has(normalize(name)));
    })
}
```

```
    .join('\n');
}
```

Extrait simplifié (TypeScript) : suppression des mentions de modèles absents des résultats des outils.

Un travail spécifique a aussi été mené sur les grades de reconditionnement. Les états peuvent être exprimés différemment selon les marchands, ce qui peut créer des confusions. Une nomenclature stricte permet de mieux encadrer les réponses de Sam.

J'ai également travaillé sur des corrections liées aux modèles trop récents ou aux variantes de modèles. Par exemple, un assistant peut confondre deux variantes proches d'un même produit. Le ciblage par modèle exact permet de réduire ce risque en visant un modèle précis.

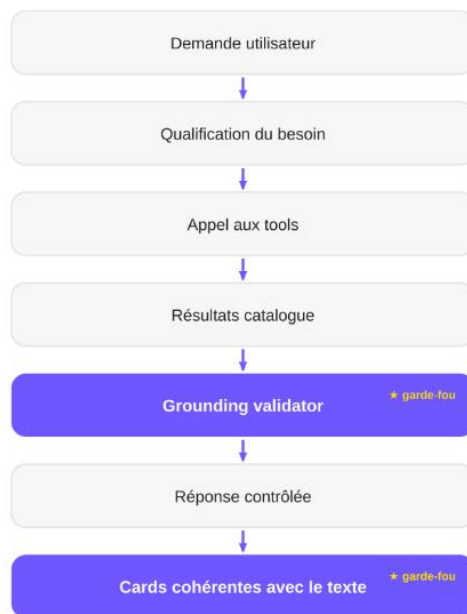


Figure 15 — Principaux garde-fous mis en place pour limiter les hallucinations de Sam.

6.6. Backoffice de suivi et analytics

Un autre chantier important a été le backoffice de suivi de Sam. Une fois l'assistant utilisé par des visiteurs, l'équipe doit pouvoir analyser son fonctionnement. Il ne suffit pas de développer l'assistant : il faut aussi comprendre comment il est utilisé.

Le backoffice permet de consulter les conversations, d'observer les actions effectuées, d'identifier les problèmes et de suivre certains indicateurs. Il permet aussi de mieux comprendre les demandes réelles des utilisateurs, qui peuvent être différentes de celles imaginées pendant le développement.

J'ai travaillé sur le suivi des actions utilisateur, le replay annoté, le funnel, la détection du device, l'analyse des interactions et le suivi du coût des modèles. Ces éléments permettent à l'équipe de prendre des décisions plus précises pour améliorer Sam.

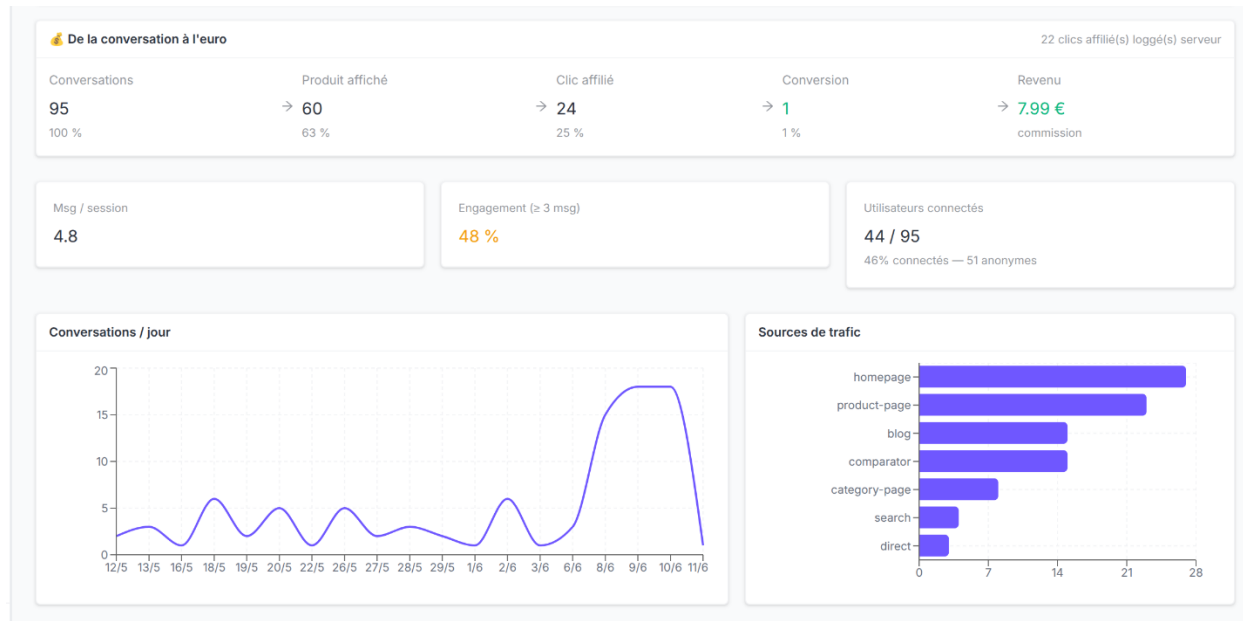


Figure 16 — Suivi de l'usage de Sam : volume de conversations, taux d'engagement et sources de trafic.

6.7. Tests, documentation et état du logiciel

En fin de développement, j'ai participé aux tests, aux corrections et à la documentation des fonctionnalités réalisées.

Les tests portaient sur plusieurs types de situations : demandes simples, demandes avec plusieurs contraintes, conversations incomplètes, recherches sans résultat, demandes ambiguës ou questions sortant du périmètre de Combak. Ces tests étaient nécessaires, car un assistant IA peut produire des réponses différentes selon la formulation de l'utilisateur.

J'ai également utilisé des commandes de vérification du projet, comme le type-check, le lint et des tests liés au matching regex. Ces vérifications permettaient de détecter certaines erreurs avant intégration.

La documentation a aussi joué un rôle important. Dans un projet qui évolue vite, il est nécessaire de conserver une trace des choix réalisés, des problèmes rencontrés et des décisions prises. Les artefacts de pilotage mis en place pour Ask Combak ont contribué à cette documentation.

À la fin de mon stage, Sam était passé d'un prototype de déclencheur de chat à un assistant plus complet, robuste et observable. Il disposait d'une interface plus aboutie, d'une logique

de recherche connectée aux données, de garde-fous anti-hallucination, d'un backoffice d'analyse et d'un suivi analytics.

Un aboutissement concret a marqué la fin de mon stage : le 21 mai 2026, Sam a été mis en production et lancé publiquement sur le site Combak. Voir passer de vraies conversations d'utilisateurs sur un produit auquel j'avais contribué, et pouvoir en suivre l'usage dans le backoffice que j'avais aidé à construire, a été la concrétisation la plus parlante de ces semaines. Sam est d'ailleurs accessible et testable en ligne, à l'adresse combak.co/ask.

En parallèle, le pipeline de déploiement de catégories avait été renforcé. Plusieurs catégories ont été ajoutées ou améliorées, et certains processus auparavant manuels ont été transformés en étapes plus reproductibles. Ce travail a permis de rendre la plateforme plus évolutive.

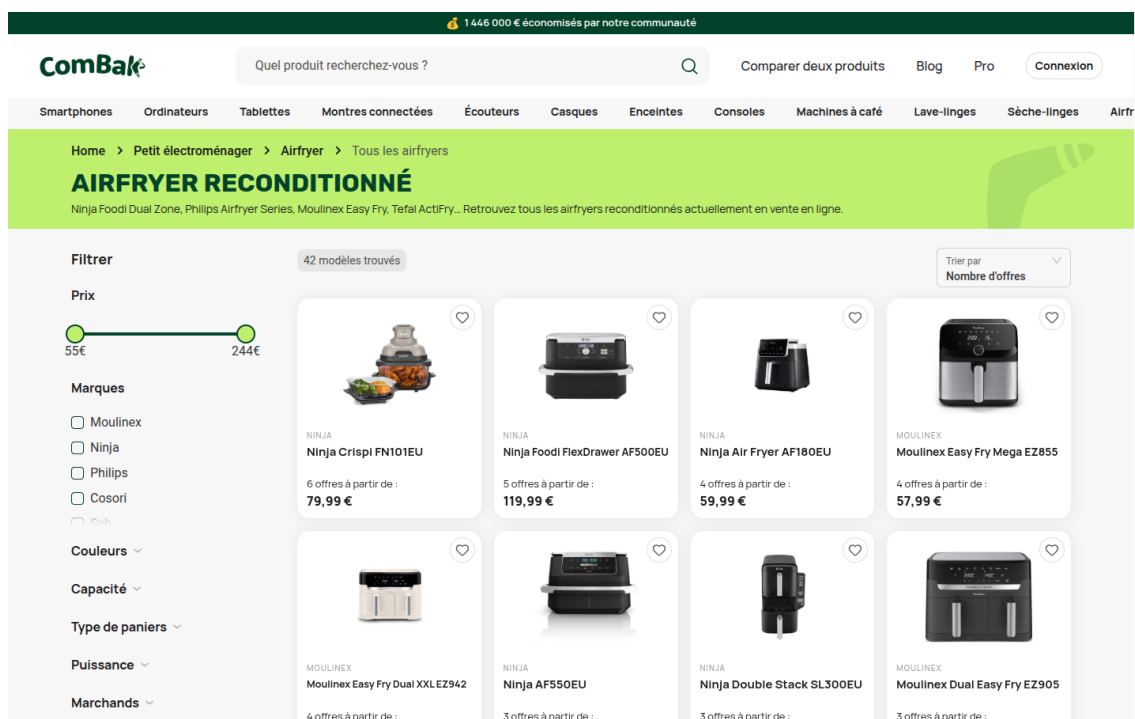


Figure 17 — La catégorie airfryer reconditionné, déployée pendant le stage.

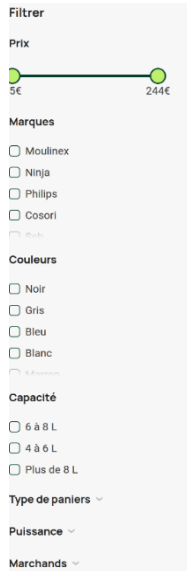


Figure 18 — Les filtres à facettes d'une page catégorie (prix, marque, couleur, capacité, puissance).

7. Résultats obtenus

7.1. Livrables réalisés

À la fin de mon stage, plusieurs livrables concrets avaient été réalisés. Le principal livrable est Sam, l'assistant IA conversationnel de Combak. Il est passé d'une première version déjà existante à un assistant plus complet, plus robuste et mieux intégré au parcours utilisateur.

Côté interface utilisateur, j'ai contribué à la refonte du déclencheur Ask Combak, à l'expérience mobile, à l'affichage des messages, aux suggestions cliquables, aux cards produits et à l'historique, ce qui rend l'assistant plus simple et plus visible. Côté serveur, j'ai travaillé sur la route API, le streaming, le function calling, la mémoire conversationnelle, les garde-fous anti-hallucination et l'accès aux données du catalogue, autant d'éléments qui fiabilisent les réponses en les fondant sur les données réelles de Combak.

Sur le suivi interne, le backoffice d'analyse, le replay des conversations, le tracking des actions, le suivi du coût des modèles et les indicateurs de funnel permettent à l'équipe de comprendre l'usage de Sam et de repérer les améliorations. S'y ajoute l'industrialisation du déploiement de catégories : cinq nouvelles catégories ont été ajoutées ou améliorées, avec un processus plus reproductible (parsers, matching regex, facettes, SEO, maillage interne). J'ai enfin mené des contributions transverses : intégration du partenaire AfB, refonte de pages univers, amélioration des filtres, suppression de flashes visuels et corrections d'interface.

Livrable	Description	État
Interface Ask Combak	Déclencheur, chat, mobile, suggestions, cards produits	Réalisé
Route API de chat	Traitement des messages, streaming, orchestration	Réalisé
Function calling	Accès contrôlé aux données catalogue	Réalisé
Mémoire conversationnelle	Conservation du contexte utile	Réalisé
Garde-fous	Réduction des hallucinations et incohérences	Réalisé
Backoffice	Analyse des conversations, replay, dashboard	Réalisé
Tracking analytics	Actions utilisateur, device, funnel, coûts	Réalisé
Catégories produit	Casques, café, lave-linge, sèche-linge, airfryer	Réalisé
Partenaire AfB	Intégration du flux partenaire	Réalisé
Documentation et tests	Scripts, notes, artefacts de pilotage	Réalisé

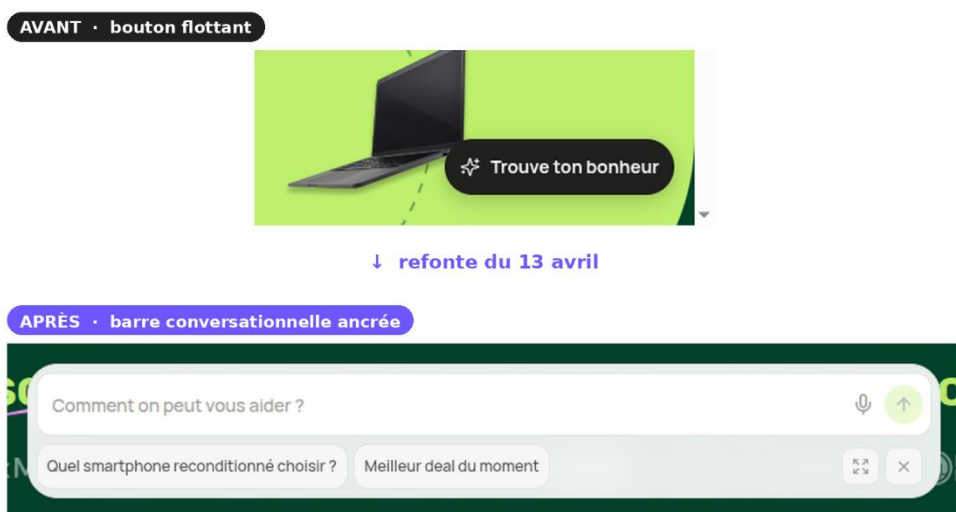


Figure 19 — Avant / après : évolution du déclencheur de Sam, du bouton flottant à la barre conversationnelle.

7.2. Comparaison avec les objectifs initiaux

Les objectifs initiaux du stage ont globalement été atteints. Sam a été amélioré sur les principaux axes prévus : expérience utilisateur, fiabilité, connexion aux données, suivi interne et extensibilité. Le projet a aussi été enrichi par des travaux transverses qui ont dépassé le périmètre strict de l'assistant.

Au 11 juin 2026, sur les trente derniers jours (95 conversations depuis le lancement du 21 mai), les indicateurs (données internes, communiquées avec l'accord de l'entreprise) atteignent ou dépassent les cibles fixées dans le PRD :

Indicateur	Mesuré (30 jours)	Cible PRD	Statut
Messages par session	≈ 4,8	3 à 5	atteint
Engagement (≥ 3 messages)	≈ 48 %	> 15 %	largement dépassé
Modèles / offres affichés	≈ 63 % des conversations	—	—
Clic sur une offre	≈ 25 % des conversations (40 % de celles ayant vu une offre)	> 25 %	atteint
Coût LLM moyen / conversation	≈ 0,0164 \$ (1,44 € sur 30 jours)	maîtrisé	validé

Dès les premières semaines, l'ensemble des cibles du PRD est atteint ou dépassé : l'engagement (48 %) dépasse largement la cible (plus de trois fois), le taux de clic sur une offre atteint 25 % (au niveau de la cible visée), et le coût par conversation reste de l'ordre du centime, ce qui valide le choix de l'orchestration multi-modèle. Le revenu d'affiliation généré couvre déjà le coût des modèles de langage.

Objectif initial	Résultat obtenu	Commentaire
Améliorer l'expérience utilisateur	Atteint	Interface plus claire, meilleure adaptation mobile, suggestions et cards
Connecter Sam aux données réelles	Atteint	Utilisation du fonction calling et des outils reliés au catalogue
Réduire les hallucinations	Atteint en grande partie	Garde-fous, grounding, cohérence cards/texte, validation des budgets
Conserver le contexte de conversation	Atteint	Mise en place d'un état conversationnel plus simple et maintenable
Suivre l'usage de Sam	Atteint	Backoffice, replay, tracking, funnel, coût des modèles

Préparer l'extension à plusieurs catégories	Atteint	Logique category-agnostic et ajout de plusieurs catégories
Industrialiser les catégories	Atteint partiellement / en progression	Pipeline amélioré, mais encore perfectible selon les catégories
Stabiliser le produit	Atteint partiellement	Fonctionnel et intégré, mais améliorable par itérations futures

Le travail réalisé montre aussi que certains objectifs ont évolué pendant le stage. Par exemple, l'observabilité et le suivi du coût des modèles sont devenus des sujets importants au fur et à mesure que Sam se rapprochait d'un usage réel. De même, les travaux sur les catégories ont pris de l'importance car ils rendaient la plateforme plus complète et donnaient à Sam davantage de données exploitables.

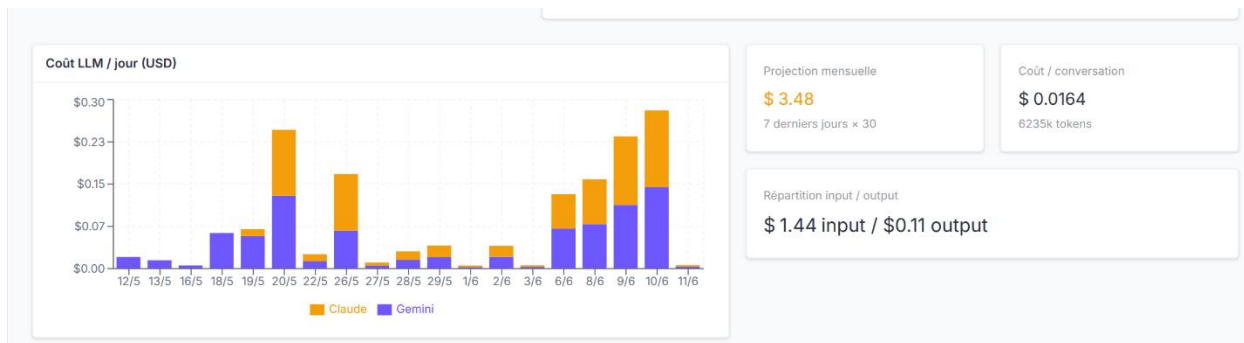


Figure 20 — Suivi du coût des modèles de Sam : coût LLM par jour, projection mensuelle et coût par conversation.

7.3. Évolutions proposées

Même si les principaux objectifs ont été atteints, plusieurs évolutions peuvent encore être envisagées.

L'une de ces évolutions concerne l'évaluation automatique des réponses : tester Sam plus systématiquement sur des scénarios représentatifs (recherche avec budget, demande ambiguë, comparaison, absence de résultat, question hors périmètre) permettrait de repérer vite les régressions après une modification. Une autre porte sur la mémoire conversationnelle, qui pourrait s'étendre à plusieurs sessions, en restant prudent sur la confidentialité et la pertinence des informations conservées. L'extension à davantage de catégories est aussi un chantier ouvert, chaque famille de produits pouvant apporter ses propres critères, facettes et vocabulaire.

Le coût des modèles mérite également un travail continu, pour identifier les appels indispensables, simplifier certains traitements et choisir le modèle le plus adapté à chaque

tâche. L'impact écologique pourrait être davantage mis en avant dans les recommandations, en cohérence avec le positionnement de Combak sur le reconditionné. Le backoffice, enfin, pourrait être enrichi pour faciliter l'analyse qualité : taguer plus facilement les conversations problématiques, suivre les erreurs récurrentes ou comparer plusieurs versions du déclencheur.

7.4. Limites restantes

Malgré les avancées réalisées, certaines limites subsistent. La première tient à la dépendance aux données : Sam ne produit de bonnes recommandations que si le catalogue est fiable et bien structuré ; une offre mal rattachée ou un flux partenaire erroné le limite aussitôt. Une autre tient à l'IA elle-même, qui reste probabiliste : même avec des garde-fous, le modèle peut mal interpréter une demande ou produire une phrase à vérifier. Le travail de fiabilisation réduit ce risque sans le supprimer totalement.

D'autres limites tiennent à l'usage. Les demandes très vagues, comme « je veux un bon téléphone pas cher », obligent à trouver le bon équilibre entre poser des questions et proposer vite des pistes, un compromis difficile à régler parfaitement. Le coût grandit aussi avec l'usage, même s'il est suivi de près. Enfin, certaines catégories ont des critères très particuliers, et beaucoup d'améliorations demandent du recul sur les vraies conversations : Sam doit rester un produit évolutif, inscrit dans une boucle d'amélioration continue.

8. Acquis de compétences

Avant de détailler mes acquis, le tableau ci-dessous relie le travail réalisé pendant le stage aux compétences du référentiel BUT Informatique.

Compétence (référentiel BUT Informatique)	Mise en œuvre pendant mon stage
Réaliser un développement d'application	Développement full-stack de Sam : interface conversationnelle, route API, function calling, mémoire et garde-fous (React/Next.js, TypeScript).
Optimiser des applications	Suivi et maîtrise du coût des modèles, streaming des réponses, corrections de performance front-end (flashs visuels, CLS).
Administrer des systèmes informatiques communicants complexes	Déploiement serverless sur Vercel (fonctions, crons) et intégration des flux partenaires dans le pipeline de données.
Gérer des données de l'information	MongoDB et Typesense : indexation, matching regex, normalisation des marques et déploiement de nouvelles catégories.
Conduire un projet	Travail itératif à partir d'un cadrage produit (PRD), artefacts de pilotage et suivi hebdomadaire sous Notion.

Collaborer au sein d'une équipe informatique	Git et GitHub (branches, pull requests, revues de code) et co-développement quotidien avec mon maître de stage.
--	---

8.1. Compétences techniques

Ce stage m'a permis de développer de nombreuses compétences techniques.

J'ai d'abord progressé en développement full-stack TypeScript. J'ai travaillé sur une application Next.js avec React, des routes API, des composants d'interface, du streaming, des styles SCSS et une architecture de production.

J'ai également acquis des compétences en ingénierie IA appliquée. J'ai travaillé sur l'orchestration de modèles de langage, le function calling, la mémoire conversationnelle, les prompts, le streaming, les garde-fous anti-hallucination et l'observabilité des coûts. Ces notions étaient nouvelles pour moi et m'ont permis de mieux comprendre comment intégrer un modèle génératif dans un produit réel.

J'ai aussi progressé sur les bases de données et les moteurs de recherche. Typesense m'a permis de mieux comprendre les collections, les requêtes, les facettes et la recherche orientée catalogue. MongoDB m'a servi à manipuler des données plus flexibles : conversations, analytics, règles de matching ou informations internes.

Le travail sur les catégories m'a permis de découvrir des problématiques de pipeline de données : parsing de flux marchands, normalisation, matching regex, agrégation de modèles et génération de pages catalogue. Ces sujets m'ont montré que la qualité d'un produit web dépend fortement de la qualité des données en amont.

Enfin, j'ai découvert des notions liées à la performance front-end, comme l'hydratation*, les flashes visuels et le CLS*. Ces problèmes sont parfois peu visibles en développement local, mais importants dans une application en production.

8.2. Compétences méthodologiques

Sur le plan méthodologique, ce stage m'a appris à travailler dans une base de code existante. Cela demande une approche différente d'un projet commencé de zéro. Avant de modifier un fichier, il faut comprendre pourquoi il existe, quelles parties du projet en dépendent et quelles conventions il respecte.

J'ai aussi appris à travailler à partir d'un cadrage produit. Le PRD* donnait une direction, mais les choix concrets de développement demandaient des arbitrages. Il fallait transformer des objectifs produit en tâches techniques, puis vérifier que les développements répondaient bien au besoin.

Le travail par itérations, central sur ce projet (méthode détaillée en 4.1), m'a appris à avancer par essais successifs : tester, accepter de revenir sur une décision, puis améliorer, plutôt que de tout figer à l'avance.

J'ai également progressé dans l'utilisation de Git et GitHub en contexte professionnel : branches, commits, pull requests, revues de code, résolution de conflits et intégration progressive. Ces pratiques m'ont appris à produire un code plus lisible et à mieux justifier mes choix.

Enfin, j'ai compris l'importance de la documentation. Les artefacts de pilotage, les résumés d'itération, les checklists et les notes techniques permettent de conserver une trace des décisions. Dans un projet qui évolue vite, cette mémoire est essentielle.

8.3. Compétences humaines

Travailler dans une équipe aussi petite change beaucoup de choses. Mon maître de stage, Tom Rouchy, était à la fois le CEO et le développeur principal. Mes questions trouvaient une réponse rapidement, et mes pull requests étaient relues par la personne qui connaissait le mieux le projet. La collaboration s'est très bien passée : on a vraiment fonctionné comme une équipe, et c'est ce travail à deux sur Sam qui m'a fait progresser le plus vite.

J'ai surtout appris à ne pas rester bloqué seul trop longtemps : préparer ma question, montrer ce que j'avais déjà essayé, dire clairement où ça coïncait. Le périmètre du stage a aussi beaucoup bougé (Sam, mais aussi les catégories, les filtres, le tracking, un partenaire), ce qui m'a appris à m'adapter et à passer d'un sujet à l'autre en gardant de la cohérence. Dans une startup, une décision technique a un impact direct sur le produit et les utilisateurs, et c'est exactement ce qui m'a plu.

8.4. Formation acquise pendant le stage

La principale formation acquise pendant le stage concerne l'usage professionnel de l'intelligence artificielle dans le développement. J'ai utilisé des outils comme Claude Code dans Cursor, ainsi que d'autres assistants IA pour comprendre du code, générer des pistes, refactorer, documenter ou automatiser certaines tâches.

Cette manière de travailler était nouvelle pour moi. Elle n'est pas encore au cœur de la formation classique, mais elle devient importante dans les métiers du développement. J'ai appris à formuler des demandes précises, à découper un problème, à relire le code proposé, à corriger les erreurs et à intégrer ces outils dans un flux de production réel.

J'ai également été formé indirectement à des notions d'ingénierie LLM* : prompts, tool calling, grounding, mémoire, streaming, coûts, observabilité et validation des réponses. Ces notions m'ont permis de mieux comprendre les différences entre un prototype IA et une fonctionnalité IA utilisable dans un produit.

Enfin, j'ai appris des pratiques professionnelles liées à la qualité de code : typage strict, tests, lint, revues, documentation, confidentialité et sécurité. Ces pratiques sont importantes pour passer d'un projet fonctionnel à un projet maintenable.

8.5. Difficultés rencontrées

La difficulté que je retiens le plus, c'est d'avoir tourné en rond sur Sam. Au début, je pensais boucler ma partie du chatbot en deux semaines environ, la réalité a été tout autre. Un assistant conversationnel ne se comporte pas comme un programme classique : une même modification de prompt, d'outil ou de garde-fou pouvait corriger un cas et en casser un autre. J'avancais, je testais, je revenais en arrière ; il a fallu accepter ce travail par essais successifs plutôt que de tout prévoir d'avance.

La fiabilité a été le cœur du problème. Un assistant peut écrire une phrase parfaitement fluide... et fausse. J'ai dû corriger des cas où Sam mélangeait deux variantes d'un même modèle, ou citait un produit absent des résultats réels. C'est là que j'ai travaillé sur le ciblage exact du modèle et sur un validateur qui supprime toute mention d'un produit qui n'existe pas au catalogue.

Tous les bugs n'étaient pas « intelligents » pour autant : un popup mal affiché sur Safari mobile, des flashes visuels au chargement d'une page... le genre de détail invisible en local et évident en production.

Au début, j'étais un peu perdu : la base de code était grande et Sam déjà entamé. Vers la fin, j'étais devenu autonome sur mes chantiers, c'est sans doute la progression dont je suis le plus fier. J'ai aussi appris à utiliser des outils comme Claude Code sans confiance aveugle : relire, simplifier, tester.

9. Conclusion

Ce stage chez Combak m'a plongé dans un produit réel, utilisé en conditions professionnelles. Ma mission principale a porté sur Sam, l'assistant IA conversationnel de recommandation de produits reconditionnés. J'ai contribué à son amélioration, à sa fiabilisation et à son intégration dans le parcours utilisateur.

Le travail réalisé a couvert plusieurs aspects : interface conversationnelle, expérience mobile, route API, streaming, function calling, mémoire conversationnelle, garde-fous anti-hallucination, backoffice, tracking analytics, tests et documentation. En parallèle, j'ai participé à l'industrialisation du déploiement de nouvelles catégories produit et à plusieurs contributions transverses, comme l'intégration du partenaire AfB ou l'amélioration des filtres.

Ce stage m'a permis de mieux comprendre la différence entre un projet scolaire et un projet professionnel. Dans un projet réel, le code doit être maintenable, testé, relu et intégré à un existant. Il faut aussi tenir compte de contraintes produit, économiques et humaines. Dans le cas de Sam, les enjeux dépassaient largement la génération d'une réponse par IA : il fallait garantir la fiabilité, l'accès aux données réelles, le suivi des coûts et l'amélioration continue.

Sur le plan technique, j'ai développé des compétences en full-stack TypeScript, en architecture web, en manipulation de données, en recherche avec Typesense, en MongoDB et en ingénierie IA appliquée. Sur le plan méthodologique, j'ai appris à travailler par itérations, à utiliser Git dans un contexte collaboratif, à documenter mes choix et à accepter les retours de revue de code. Sur le plan humain, j'ai gagné en autonomie, en communication et en capacité d'adaptation.

Ce stage a aussi confirmé mon intérêt pour les projets mêlant développement web, données et intelligence artificielle. Il m'a montré que l'IA peut être un outil puissant, mais qu'elle doit être encadrée par des règles, des données fiables, des tests et une supervision humaine. La compétence la plus importante que je retiens n'est donc pas seulement de savoir utiliser un modèle de langage, mais de savoir l'intégrer correctement dans un produit.

Enfin, cette expérience m'a appris à prendre des responsabilités dans une base de code existante. Construire un projet depuis zéro est formateur, mais savoir comprendre, améliorer et maintenir un projet déjà en production est une compétence tout aussi essentielle pour un développeur.

10. Usage de l'intelligence artificielle chez Combak

Combak utilise très fréquemment l'intelligence artificielle dans le domaine informatique. L'IA est présente à deux niveaux : comme sujet produit, avec Sam et les outils internes liés aux modèles de langage, et comme outil de développement utilisé au quotidien.

Pour la production de code, l'équipe utilise principalement Claude Code intégré à Cursor, ainsi que ChatGPT pour certains échanges, recherches ou reformulations. Ces outils servent à générer du code, proposer des refactorings, produire de la documentation, expliquer des parties de code existantes ou automatiser certaines tâches répétitives.

L'IA est donc utilisée spécifiquement pour la production de code, mais le code généré n'est pas intégré automatiquement. Il doit être relu, adapté, testé et validé. Pendant mon stage, j'ai régulièrement repris du code proposé par l'IA, soit manuellement, soit en reformulant une demande pour obtenir une correction. Cette pratique m'a appris à considérer l'IA comme un assistant de développement, et non comme une source infaillible.

L'entreprise investit dans ces outils, à travers des abonnements et des coûts d'API. Ces coûts sont aussi suivis dans le cadre du produit Sam, car l'utilisation de modèles de langage peut avoir un impact économique. Le suivi par modèle permet de mieux comprendre le coût réel de l'assistant.

L'usage de l'IA est encouragé et encadré par des pratiques internes. Il existe notamment des commandes, des workflows et des consignes de confidentialité. Les informations sensibles, comme les secrets, les données personnelles, les contrats partenaires, les prompts système ou les données business confidentielles, ne doivent pas être transmises librement aux outils d'IA.

Il n'y a pas eu de formation formelle longue pendant mon stage, mais j'ai été formé par la pratique, les échanges avec mon maître de stage et les revues de code. J'ai appris à formuler des demandes efficaces, à vérifier les réponses, à corriger le code généré et à intégrer ces outils dans un flux de production.

Chaque développeur peut utiliser l'IA selon ses besoins, mais dans un cadre de responsabilité : il doit comprendre ce qu'il intègre, vérifier les effets du code et respecter les règles de confidentialité. L'IA accélère certaines tâches, mais elle ne remplace pas la compréhension technique ni la responsabilité du développeur.

Ce stage m'a donc permis de découvrir une pratique professionnelle de l'IA appliquée au développement. J'en retiens que ces outils sont très utiles pour gagner du temps, explorer des solutions ou automatiser des tâches répétitives, mais qu'ils nécessitent une relecture rigoureuse. Leur intérêt dépend surtout de la capacité du développeur à poser les bonnes questions, à évaluer les réponses et à conserver la maîtrise du code produit.

Glossaire

A/B test : méthode consistant à comparer deux versions d'une interface ou d'une fonctionnalité afin de mesurer laquelle fonctionne le mieux auprès des utilisateurs.

Affiliation : modèle économique dans lequel une plateforme peut être rémunérée lorsqu'elle redirige un utilisateur vers un partenaire et qu'une action est réalisée, par exemple un achat.

Analytics first-party : système de mesure reposant sur des données collectées directement par le site, plutôt que par un service externe tiers.

API : interface permettant à deux parties d'un logiciel ou à deux applications de communiquer entre elles.

Backoffice : interface interne utilisée par l'équipe pour administrer, suivre ou analyser une fonctionnalité.

Card produit : composant d'interface présentant un produit ou une offre sous une forme visuelle synthétique.

Category-agnostic : se dit d'une logique conçue pour fonctionner avec plusieurs catégories de produits, sans être codée spécifiquement pour une seule catégorie.

CLS : indicateur lié à la stabilité visuelle d'une page web. Il mesure les déplacements inattendus d'éléments pendant le chargement.

Commit : enregistrement d'une modification dans un dépôt Git.

CTA : bouton ou lien incitant l'utilisateur à effectuer une action, par exemple cliquer sur une offre.

Drawer : panneau latéral ou inférieur qui s'ouvre dans une interface, souvent utilisé sur mobile.

Facette : critère de filtrage utilisé dans un moteur de recherche, par exemple la marque, le prix ou l'état.

Function calling : mécanisme permettant à un modèle de langage d'appeler des fonctions définies par l'application pour obtenir des données ou effectuer une action.

Grade : état d'un produit reconditionné, par exemple très bon état, bon état ou état correct.

Grounding : principe consistant à ancrer une réponse d'IA dans des données vérifiables, plutôt que dans les seules connaissances du modèle.

Hallucination : réponse générée par une IA qui semble crédible mais contient une information fausse, inventée ou non vérifiée.

Hydratation : étape où une page générée côté serveur devient interactive côté navigateur.

LLM : grand modèle de langage capable de comprendre et générer du texte.

Matching regex : méthode utilisant des expressions régulières pour associer une offre marchand à un modèle produit.

MongoDB : base de données orientée documents, utilisée pour stocker des données flexibles.

Next.js : framework web basé sur React, utilisé pour créer des applications web modernes.

PRD : Product Requirements Document, document de cadrage décrivant les objectifs, les besoins et les contraintes d'un produit.

Prompt : instruction donnée à un modèle d'intelligence artificielle pour orienter sa réponse.

Reconditionné : produit déjà utilisé, vérifié, réparé si nécessaire, puis remis en vente.

SEO : optimisation d'un site pour améliorer sa visibilité dans les moteurs de recherche.

Streaming : affichage progressif d'une réponse au fur et à mesure de sa génération.

Tool calling : synonyme de function calling.

Tracking : suivi des actions effectuées par les utilisateurs, par exemple les clics ou ouvertures d'un chat.

Typesense : moteur de recherche utilisé pour indexer et rechercher rapidement des données.

Vercel : plateforme d'hébergement et de déploiement d'applications web.

Zod : bibliothèque TypeScript permettant de valider des structures de données.

Table des figures

Figure 1 — Page d'accueil de Combak, donnant accès aux différents univers de produits reconditionnés

Figure 2 — Page d'une catégorie Combak (ordinateurs portables reconditionnés) avec filtres et offres comparées

Figure 3 — Pipeline simplifié des données Combak, depuis les flux partenaires jusqu'au catalogue et à Sam

Figure 4 — Déclencheur Ask Combak intégré au site Combak

Figure 5 — Conversation avec Sam : qualification du besoin puis recommandation de smartphones reconditionnés

Figure 6 — Dashboard backoffice de Sam : KPIs conversationnels, funnel d'engagement et suivi des coûts

Figure 7 — Deux catégories déployées pendant le stage : sèche-linge et airfryer reconditionnés

Figure 8 — Organisation chronologique des principaux chantiers du stage

Figure 9 — Architecture générale de Sam : interface, route API, modèles IA, outils et données

Figure 10 — Flux de traitement d'un message utilisateur dans Sam

Figure 11 — Exemple simplifié d'état conversationnel utilisé par Sam

Figure 12 — Interface mobile de Sam après refonte de l'expérience conversationnelle

Figure 13 — Rôle de la route API dans le traitement d'un message Sam

Figure 14 — Recommandation d'ordinateurs portables par Sam, avec des cards issues du catalogue réel

Figure 15 — Principaux garde-fous mis en place pour limiter les hallucinations de Sam

Figure 16 — Suivi de l'usage de Sam : volume de conversations, taux d'engagement et sources de trafic

Figure 17 — La catégorie airfryer reconditionné, déployée pendant le stage

Figure 18 — Les filtres à facettes d'une page catégorie (prix, marque, couleur, capacité, puissance)

Figure 19 — Avant / après : évolution du déclencheur de Sam, du bouton flottant à la barre conversationnelle

Figure 20 — Suivi du coût des modèles de Sam : coût LLM par jour, projection mensuelle et coût par conversation

Annexes

Annexe 1 — Tableau chronologique simplifié des principales contributions

Période	Contributions principales
13 au 17 avril	Déclencheur Ask Combak, page /ask, premières vues backoffice, garde-fous, responsive

20 au 24 avril	Architecture hybride, anti-hallucination, refonte category-agnostic, extraction d'état
22 au 28 avril	Cohérence cards/texte, ciblage des modèles, refonte mobile, sanitizer typographique
29 avril au 5 mai	Qualification du besoin, polish pré-production, rate limits, amélioration du grounding
12 au 28 mai	Tracking, replay annoté, dashboard, détection device, A/B test
Fin mai à début juin	Catégories lave-linge, sèche-linge, airfryer, AfB, corrections finales

Annexe 2 — Exemples de scénarios de test Sam

Scénario	Objectif du test
"Je cherche un smartphone à moins de 400 €"	Vérifier la recherche avec budget
"Je veux un ordinateur pour les cours"	Vérifier la qualification du besoin
"Et avec une bonne autonomie ?"	Vérifier la mémoire conversationnelle
"Compare ces deux modèles"	Vérifier la logique de comparaison
"Je veux le dernier modèle sorti"	Vérifier les limites de connaissance et le grounding
Question hors périmètre	Vérifier le recentrage vers Combak
Recherche sans résultat	Vérifier la réponse prudente

Annexe 3 — Confidentialité

Certains éléments ne doivent pas apparaître dans le rapport ou doivent être floutés dans les captures :

- emails utilisateurs ;
- identifiants de conversations ;
- clés API ;
- prompts système complets ;
- contrats partenaires ;
- chiffres de conversion confidentiels ;
- données personnelles ;
- conversations réelles non anonymisées ;
- coûts internes précis si l'entreprise ne souhaite pas les publier.

Annexe 4 — Tableau détaillé des commits

Historique des commits — Stage Combak

101 commits réalisés sur la période (13 avril → 8 juin 2026) - extrait représentatif ci-dessous

DATE	COMMIT	TYPE	DESCRIPTION
05/06	eec7c257	chore	hero script de normalisation des images hero Darty (strip Pad → produit qui remplit le cadre)
04/06	6883a96f	fix	ask-combak restaure la colonne Interactions + email au hover Auth (analyse)
01/06	7714f879	fix	tumble-dryers noms modèles propres + retrait du type de séchage de l'affichage
28/05	b128e542	chore	branch-snapshot tumble-dryers category + ask-combak iterations
27/05	e36957b8	fix	preserve ask combak source context
22/05	bd4787db	docs	generate-category-assets fiabilise la génération des logos marques (Section 3)
21/05	828cc593	feat	ask-combak refonte funnel UI + dashboard détails actions
21/05	9e09f7e6	fix	ask-combak corrections review backoffice + tracking interactions
20/05	ed3c7b7c	feat	coffee-machines integrate category under petit-electromenager universe
19/05	a055e06a	refactor	ask-combak retire le \$facet triggerBreakdown mort de stats.ts
19/05	7006ec6a	feat	universe redesign sections univers (layouts landing, boomerang, hiérarchie SEO)
18/05	859752b4	feat	ask-combak indicateur ouverture chat (suggestion vs recherche)
12/05	7e8e3d50	fix	model-regex 2-pass recompute + camelCase JSON support + skill validation
07/05	fd9bb37f	fix	ui popup reset password polish + zone cliquable étendue marchand mobile
05/05	f7816df2	fix	ask-combak harden grounding, polish UI, fix portal click leak
30/04	65bb66cd	feat	ask-combak refine trigger & suggestions UI
29/04	8fe747ec	refactor	ask-combak qualify-vs-search rule based on axis count, no privileged axis
29/04	3225357c	feat	ask-combak search_models on first qualification signal
28/04	30b5fb3d	feat	seo add headphone (casque) category SEO content
28/04	750dab3b	fix	ask-combak force fresh conversation when trigger sends a new question
28/04	1ccc57f7	style	ask-combak bump floating pill size for better tap target
28/04	ec2edc80	fix	ask-combak éviter d'extraire et de forcer un reseller depuis une simple question (Bugs A+C)
28/04	cac5b8ba	feat	ask-combak usage-aware ranking from radar scores (strictly category-agnostic)

27/04	34510800	style	ask-combak tighten conversation history layout (sidebar mode)
27/04	09f66a4f	autre	Revert "feat(ask-combak): add tech metrics dashboard section (latency, quality, robustness, tokens)"
27/04	c72011ed	fix	ask-combak enforce numeric coherence between minBudget and maxBudget
27/04	6a9a93e8	fix	ask-combak isolate new conversations + invalidate history cache on creation
24/04	0093f57b	feat	ask-combak state LLM Haiku + contradictions + budgetTier qualitatif
23/04	2d5bb696	feat	ask-combak hardening scalable — cards=mentions + prompt anti-hallucination + intent simplifié
22/04	dfc9027f	feat	ask-combak iteration 001 — grades nomenclature triple garde-fou
20/04	e29bd489	feat	ask-combak hybrid architecture — deterministic intent classifier + post-filter
17/04	e4f81ab7	feat	ask-combak merge search_models cards + multi-brand diversification
16/04	7963b9d5	feat	ask-combak voice input, rate limit peek, sentiment data, UI polish
13/04	9638b83a	feat	ask-combak redesign trigger as floating input bar + dedicated /ask page

Résumé / Abstract et mots-clés

Résumé

Ce rapport présente le stage que j'ai effectué chez Combak, une startup rennaise du reconditionné et de la seconde main (électronique et électroménager pour l'instant, d'autres univers à venir). J'ai principalement contribué au développement de Sam, l'assistant IA conversationnel de Combak.

Pendant ce stage, j'ai travaillé sur l'interface conversationnelle, l'expérience mobile, la route API de streaming, le function calling, la mémoire conversationnelle, les garde-fous anti-hallucination, le backoffice d'analyse et le tracking analytics. J'ai également participé à l'industrialisation du déploiement de nouvelles catégories produit, ainsi qu'à plusieurs contributions transverses comme l'intégration d'un partenaire et l'amélioration des filtres.

Ce stage m'a permis de développer des compétences en développement full-stack TypeScript, en architecture web, en manipulation de données, en intelligence artificielle générative et en méthodologie produit. Il m'a aussi appris à travailler dans une base de code existante, à utiliser l'IA comme outil de développement avec un regard critique et à prendre en compte les contraintes d'un produit réel.

Mots-clés : intelligence artificielle, assistant conversationnel, reconditionné, développement web, recommandation

Abstract

This report presents my internship at Combak, a startup building a comparison platform for refurbished and second-hand products (currently electronics and home appliances, with planned expansion to other categories). My main mission was to contribute to the development of Sam, Combak's conversational AI assistant for product recommendation.

During this internship, I worked on the conversational interface, mobile experience, streaming API route, function calling, conversational memory, anti-hallucination safeguards, analysis backoffice and analytics tracking. I also contributed to the industrialization of new product category deployment, as well as several cross-functional improvements such as partner integration and filter enhancements.

This internship allowed me to develop skills in full-stack TypeScript development, web architecture, data processing, generative artificial intelligence and product methodology. It also taught me how to work within an existing codebase, use AI tools for development with a critical mindset and take into account the constraints of a real product.

Keywords: artificial intelligence, conversational assistant, refurbished products, web development, recommendation